

Reactjs Indepth Explanations

In this introduction we will learn.,

1. Some Background about Reactjs
2. What is Reactjs ?
3. Features of Reactjs / Why its popular now a days
4. Single page vs Multi page applications
5. Reconciliation Process, DOM & Virtual DOM

7 Interview questions & answers from these topic

Why ReactJS was introduced?

Initially, Facebook was using **PHP for backend** and **jQuery, Javascript for frontend updates**.

As the number of users increased massively, they started facing some major problems:

- **Performance issues** - due to frequent DOM updates
- **Sharing data** between pages
- **Ui Rerenderings**
- Code became **hard to maintain**

To solve these problems, Facebook internally created a **JavaScript-based UI library called ReactJS**.

React helped them improve performance and manage UI updates efficiently.

Later, Facebook made ReactJS **open source in 2013**.

Today, ReactJS is very popular and widely used in **enterprise-level applications**.

What is ReactJS?

ReactJS is a **JavaScript-based UI library** used to build **interactive, complex, and scalable user interfaces** in an easier and more efficient way.

Library:

ReactJS is called a **library** because it provides **pre-written code and built-in functionalities** that help us build user interfaces without writing everything from scratch.

Example:

If we want to update text in the UI based on an API response:

- **In plain JavaScript:**

We manually find the element and update it.

- **In ReactJS:**

We use `useState` or `setState`, and React automatically updates the UI.

Why ReactJS is popular nowadays?

- Used to build **Single Page Applications (SPA)**
 - Follows **Component-based architecture**, which makes applications easier to build and maintain
 - Uses **JSX**
 - Uses **Virtual DOM and Reconciliation**, which improves performance
 - Large ecosystem and strong community support
-

1. What are Single Page Applications and Multi Page Applications?

General behavior:

Whenever a webpage is opened in the browser, the browser sends a request to the server for an **HTML page**.

The server responds with the HTML file along with CSS and JavaScript files.

Multi Page Applications (MPA)

- Now whenever user navigates between the pages, Each navigation sends a **request for a new HTML page**
 - Lets say user is navigating to Products page, then new request will be sent to server for Products page HTML document.
- Once server sends the response, Browser **reloads the entire page and** displays to user.
- Slower because of full page reloads

Examples:

- bbc.com
- w3schools
- Most of the Legacy websites

Single Page Applications (SPA)

- Only **one HTML page** is loaded initially
- During navigation, **no new HTML page is requested**
- UI updates only necessary parts of the application based on the **JSON data** which is received from the APIs
- No full page reload
- Faster after first load
- Smoother user experience

Examples:

- react.dev
- Facebook
- Gmail

Reconciliation & Virtual DOM?

Before learning about reconciliation & virtual dom you need to understand what is Dom.

DOM:

DOM stands for **Document Object Model**.

- When a webpage is opened, the browser converts the entire HTML document into a **tree-like structure in memory**
- This structure is called the **DOM**
- Each HTML element, attribute, and text is converted into a **node object** and these nodes will be appended to the tree,

Example HTML:

```
<div>
<h1>Hello</h1>
<button>Click me</button>
</div>
```

DOM Tree Structure:

```
div (root node)
|
|---- h1 (element node)
|      |
|      |---- "Hello" (text node)
|
|---- button (element node)
|      |
|      |---- "Click me" (text node)
```

DOM Updates in JavaScript

- JavaScript cannot directly update the webpage
- It uses the DOM as a **mediator**
- To update text, we access the DOM element and change it using `innerHTML`, `styles`, etc.
- Every DOM change causes the browser to **repaint the UI**

If there are **multiple DOM changes**, the browser redraws the page **multiple times**, which affects performance.

To resolve this performance issue, Reactjs introduced virtual dom and reconciliation process.

How Reconciliation & Virtual dom resolved this issue ?

To solve performance issues, ReactJS introduced the **Virtual DOM** and **Reconciliation process**.

How it works:

- When a React component renders, React creates a **Virtual DOM** (a copy of the real DOM) in memory
- When state or props change:
 - A **new Virtual DOM** is created
 - React compares the **old Virtual DOM** and **new Virtual DOM** using a **diffing algorithm**
 - React finds the exact changes & it will create a changeset.
 - Only those changes are applied to the **real DOM**

This process is called **Reconciliation**.

Why Virtual DOM improves performance?

- React does **not update the entire DOM**
- Only the **changed parts** of the UI are updated
- This reduces unnecessary re-rendering
- UI updates become faster and more efficient

Example without React (JavaScript):

```
const element = document.getElementById("box");  
  
element.innerHTML = "Hello updated";  
element.style.backgroundColor = "red";
```

```
element.style.border = "1px solid black";  
element.style.fontSize = "20px";
```

Here, the browser redraws the UI **4 times**.

Example with React:

```
setText("Hello updated");  
setBgColor("red");  
setBorder("1px solid black");  
setFontSize("20px");
```

Even though `setState` is called multiple times:

- React batches all updates
- Virtual DOM is updated first
- Final changes are applied to real DOM **only once**

This is why **React applications are faster and more efficient**.

Components:

- Components are building blocks of our react application.
- Each component will represent a part of UI and all components combine together to form a full fledged react application.

JSX:

- JSX means JavaScript Xml. Its a syntax extension for javascript.
- By using JSX, we can write the code similar to HTML in our javascript files. (Its HTML like code but not HTML).

Possible Interview Questions & Answers :

1. What is ReactJS?

- ReactJS is a **JavaScript-based UI library** used to build **interactive, fast, and scalable user interfaces**.
-

2. What are the features of Reactjs ?

- Reactjs is Used to build **Single Page Applications (SPA)**
 - It Follows **Component-based architecture**, which makes applications easier to build and maintain
 - It Uses **Virtual DOM and Reconciliation**, which improves performance.
 - It uses JSX.
 - It has Large ecosystem and strong community support
-

3. What are the differences between Single Page Application (SPA) & Multi page applications ?

- A Single Page Application loads **only one HTML page**.
 - Page does not reload during navigation
 - UI updates dynamically using JavaScript
 - Data is fetched using APIs (JSON)
 - **Examples:** Facebook, Gmail, react.dev
 - In a Multi Page Application:
 - Each navigation requests a **new HTML page**
 - Browser reloads the entire page
 - Slower user experience
 - **Examples:** bbc.com, w3schools, Government websites
-

4. What is DOM?

- DOM stands for **Document Object Model**.
 - Browser converts HTML into a **tree structure**
 - Each element becomes a **node**
 - JavaScript uses the DOM to update the webpage.

- DOM acts as a **bridge between JavaScript and the webpage**.
-

5. What is Reconciliation in React?

Reconciliation is the process where React:

1. Creates a new Virtual DOM after state change
 2. Compares it with the previous Virtual DOM
 3. Finds only the changed elements
 4. Updates only those changes in the real DOM
-

6. What are Components ?

- Components are building blocks of our react application.
 - Each component will represent a part of UI and all components combine together to form a full fledged react application.
-

7. What is JSX ?

- JSX means JavaScript Xml. Its a syntax extension for javascript.
 - By using JSX, we can write the code similar to HTML in our javascript files. (Its HTML like code but not HTML).
-

Environmentsetup:

In this session you are going to learn about.,

1. Node
 2. Npm
 3. Vite
 4. Folder structure explanation
 5. 16 Interview questions & Answers
-

Why Nodejs & npm are needed ?

- If you want to.,
 - Create a reactjs project (Or) if you want to run dev server (Or) if you want to install some dependencies, you need npm(Node package manager). But this npm itself comes with nodejs.
 - We use build tool like Vite/Babel. These runs on nodejs to access file system(to watch file changes), os level operations, running dev servers etc.
-

VITE:

- Vite is a build tool to create, develop and build reactjs applications.

Esbuild:

- We write our code in JSX. But browser does not understand jsx. Browser understands only javascript code. So to convert this jsx to js we use vite. Vite internally uses esbuild compiler to convert jsx to js.

Hot module reloading:

- Because of this, when ever we change some component code, vite will detect those file changes and updates this component code in browser memory.

Rollup:

- During production build Rollup Production build:
- It will optimize the code by using.,
 - Tree shaking(Removing unused code/commented code)
 - Code splitting (Breaking larger files in to smaller chunks) - if lazy loaded
 - Minification (Removing unnecessary characters, spacings, comments)
- It uses Rollup internally for creating production builds
- Command to run production build: npm run build

- Its very fast when compared to create-react-app and Webpack.
-

What happend to create-react-app ?

- Previously create-react-app was the default way to create reactjs applications
 - But its deprecated because of below reasons.
 - Slower development server.
 - Not actively maintained
 - Lot of issues with modern javascript and Reactjs features.
 - Its not completely removed. Still legacy applications uses create-react-app but modern applications should be created by using vite or other build tools.
-

Downloading Nodejs, Npm & vscode:

Download Nodejs ⇒ <https://nodejs.org/en/download/current>

Download vscode ⇒ <https://code.visualstudio.com/download>

Once its installed,

Check nodejs version ⇒ `node --version`

Check npm version ⇒ `npm --version`

Create React project :

Step 1: Create the project

```
npm create vite@latest ecommerce-app
```

- During setup, it will ask:
 - **Framework** → Select `React`
 - **Variant** → Select `JavaScript` or `TypeScript`

Step 2: Navigate to the project folder

```
cd ecommerce-app
```

Step 3: Install dependencies

```
npm install
```

- Installs React, React DOM, and Vite-specific packages.

Step 4: Run development server

```
npm run dev
```

- Starts local dev server (usually `http://localhost:5173/`)
- Instant reload when you save files

Step 5: Build for production

```
npm run build
```

- Generates optimized `dist/` folder for deployment

Step 6: Preview production build

```
npm run preview
```

- Allows you to test the production build locally

Project Structure:

Package.json:

- It contains all the metadata about the project like.,
 - Project information (name, version of project)
 - How to run, How to build the project
 - What are the dependencies, dev dependencies of this project

name:

- Its the name of your project or package you are going to build.
 - Once you publish your project, it will generate a package with this name.
-

private:

- It is a boolean (true/false) value that controls whether your package can be published to npm.
 - **true:**
 - It will Prevent the accidental publication to npm
 - It means when you do npm publish, it will not be published into npm global registry
 - Good for internal projects and When you dont want your code to publicly available.
 - **false:**
 - means when you do npm publish, it will be published into global npm registry.
 - If you want your package publicly available, then use false.
 - "version": "0.0.0":
 - This will specify the version of this project. (major.minor.patch)
 - Everytime before publishing this project, we will be updating this version.
-

"version": "0.0.0":

- This will specify the version of this project.
 - Major.minor.patch version
 - Major- If you have some breaking changes or new features.
 - Minor (1.3.0)- If some minor features are added
 - Patch (1.2.1)- some bug fixes are made.
 - Everytime before publishing this project, we will be updating this version.
-

type:"module"

In our **projects** the `"type"` field in `package.json` decides whether your project uses:

- **CommonJS (CJS)** → old Node system
- **ES Modules (ESM)** → modern JavaScript module system

CommonJS (default style)

(No `"type"` means CommonJS automatically)

Import / Export

```
// math.js
function add(a, b) {
  return a + b;
}
```

```
module.exports = add;
```

```
// index.js
const add = require('./math');
console.log(add(2,3));
```

It Uses:

- `require()`
- `module.exports`

ES Module (modern JavaScript)

Import / Export

```
// math.js
export function add(a, b) {
  return a + b;
}
```

```
// index.js
import { add } from './math.js';
console.log(add(2,3));
```

Uses:

- `import`
- `export`

Important Difference

Feature	CommonJS	ES Module
Syntax	require	import
Used in	Old Node	Modern tools (Vite, React, bundlers)

Scripts:

- You can run the scripts using npm.
- dev: vite ⇒ you can run using npm run dev ⇒ It starts vite development server on nodejs run time environment.
- build: vite build ⇒ you can run using npm run build ⇒ It will create production build files in dist folder
- lint: eslint ⇒ this will check code quality & finds bugs early during development
- preview ⇒ if we want to preview production build locally then you can use this.

- lint: eslint . ⇒ this will check code quality & helps to find bugs early during development
 - Here "Eslint ." means current directory or current folder. So eslint will scan all the files in the current directory
 - "Eslint src" means it will scan all files in src folder
 - "Eslint src/app.jsx" means it will scan app.jsx file in src folder.
 - **What Eslint will do ?**
 - It will have some predefined rules which help in.,
 - Catches bugs early
 - Keeps code consistent across team
 - Improves code quality
 - We can define our own custom rules as well.

Common rules ESLint checks:

- Missing semicolons
 - Unused variables
 - Spacing and indentation
 - Using `var` instead of `let / const`
 - Missing `===` (using `==` instead)
 - preview ⇒ if we want to preview production build locally then you can use this.
-
- We can create our own script as well.
 - Example: script : "vite".
 - Now you can run using "npm run script"
-
- dependencies:
 - These are all the dependencies your project needs during run time (means when opened in the browser/production mode)
 - devDependencies:

- These are required only during development phase. In production build these are not required.
 - Eg: npm install eslist —save-dev
-

- **Package.lock.json:**

- This will ensure all developers who are using our project has same versions of dependencies installed.
 - It contains exact version of dependencies of packages and their sub dependencies
-

- **node modules:**

- When we execute npm install command, npm will scan entire package.json file and finds the dependencies and devdependencies.
 - Now it will download all these dependencies into this nodemodules folder which can be utilized by our react project
-

- **Public folder:**

- Used to store all **Static resources that don't need processing:** (font files, images, icons etc)
 - htaccess files - used by hostinger server
 - assets - favicon, logos, icons etc
 - Static pdf's
 - index.html
-

- **Dist folder:**

- When we execute npm run build, it will build our entire project and it will create some minified file bundles and pushes them into dist folder. these files will be deployed to hosting platforms.
-

- **SRC:**

- This is the where our react code resides.
 - main.jsx:
 - This is the entry point of our application.
-

- React will start execution from here and it connects our react to html "root".

- **index.html:**

- react injects our UI to root here.

Project structure you can follow:

```
my-react-app/  
|  
├── public/  
│   └── favicon.svg  
|  
├── src/  
│   ├── assets/  
│   │   ├── images/  
│   │   └── icons/  
│   |  
│   ├── components/  
│   │   ├── common/  
│   │   │   ├── Button.jsx  
│   │   │   └── Loader.jsx  
│   │   └── layout/  
│   │       ├── Header.jsx  
│   │       └── Footer.jsx  
│   |  
│   ├── pages/  
│   │   ├── Home/  
│   │   │   ├── Home.jsx  
│   │   │   ├── HomeHero.jsx  
│   │   │   └── HomeProducts.jsx  
│   │   ├── About.jsx  
│   │   └── Contact.jsx  
│   |  
│   ├── hooks/  
│   │   └── useAuth.js  
|  
└── services/
```

```
|
| |
| | └─ api.js
| |
| | └─ utils/
| |   └─ formatDate.js
| |
| | └─ constants/
| |   └─ routes.js
| |
| | └─ styles/
| |   └─ global.css
| |
| | └─ App.jsx
| |   └─ main.jsx
|
| └─ index.html
| └─ package.json
| └─ vite.config.js
| └─ README.md
```

Interview questions & Answers:

1. What is Node.js and why is it required for React?

Answer:

Node.js is a JavaScript runtime that allows JavaScript to run outside the browser.

React development requires Node.js because:

- Build tools like **Vite, Babel, Webpack** run on Node
- npm comes bundled with Node
- Dev servers, file watching, and build processes depend on Node APIs

React apps **run in the browser**, but **React tools run on Node.js**.

2. What is npm?

Answer:

npm (Node Package Manager) is used to:

- Install third-party libraries (react, axios, mui, etc.)
- Manage project dependencies
- Run scripts like `npm run dev` and `npm run build`

npm comes **automatically installed with Node.js**.

3. What are the differences between Node.js and npm?

Answer:

Node.js	npm
JavaScript runtime	Package manager
Executes JS outside browser	Installs & manages packages
Required to run build tools	Required to manage dependencies

4. What is Vite? What are the features of vite ?

Answer:

Vite is a modern **build tool** used to:

- Create React projects
- Run development servers
- Build optimized production bundles

It is faster than Create React App and Webpack.

Features:

- Esbuild compiler - converts jsx to js
- Run development server
- Hot module replacement
- Rollup

5. What is JSX and how does Vite handle it?

Answer:

JSX is a syntax extension for JavaScript used in React. Browsers don't understand JSX.

Vite uses **esbuild** to convert JSX → JavaScript and sends JavaScript to the browser

6. What is Hot Module Replacement (HMR)?

Answer:

HMR updates only the modified component in the browser memory **without reloading the entire page**.

7. What happens when you run `npm run dev` ?

Answer:

- Node.js starts Vite
- Vite starts a dev server
- Files are watched for changes
- JSX is compiled in memory
- Browser loads the app

8. What happens when you run `npm run build` ?

Answer:

- Vite uses Rollup
- JSX → JS conversion
- Code is optimized:
 - Tree shaking

- Minification
- Code splitting
- Output is generated in `dist/` folder

9. What is Tree Shaking?

Answer:

Tree shaking removes **unused imports/code** from the final build to reduce bundle size.

Example:

```
import { add, subtract } from './math';
```

If `subtract` is unused → removed from production build.

10. Difference between dependencies and devDependencies?

Answer:

dependencies	devDependencies
Needed at runtime	Needed only during development
Included in production	Not included in production
Example: react	Example: vite, eslint

11. What is package.json?

Answer:

It is the **configuration file** of a project that contains:

- Project metadata
- Scripts
- Dependencies
- Versioning
- Module type

12. What does `"private": true` mean in package.json?

Answer:

- It will prevent accidental publishing to npm registry
- `npm publish` will fail if private is true

Used in internal or enterprise projects.

13. What is package-lock.json?

Answer:

It contains exact versions of installed dependencies. It ensures that same version of dependencies are installed for all developers.

14. What is `main.jsx` ?

Answer:

`main.jsx` is the **entry point** of the React app.

It:

- Creates React root
 - Attaches React to HTML `#root`
 - Starts rendering App component
-

15. Why should node_modules not be pushed to Git?

Answer:

- Because they are very large & can be generated using npm install.
-

16. What is Rollup ?

Answer:

- It is used internally by vite to create production builds
 - Command: npm run build
 - It will optimize the code by using.,
 - Tree shaking(Removing unused code/commented code)
 - Code splitting (Breaking larger files in to smaller chunks) - if lazy loaded
 - Minification (Removing unnecessary characters, spacings, comments)
-

Mini Assignment:

1. Create basic project using vite and understand the folder structure.

JSX:

This pdf contains explanation for.,

- JSX
 - With & without JSX difference
 - Fragments
 - Dynamic js Expressions
 - 5 Interview questions
 - 1 Mini Assignment
-

Explanation:

- JSX means JavaScript Xml. Its a syntax extension for javascript.

- By using JSX, we can write the code similar to HTML in our javascript files. (Its HTML like code but not HTML).

With out JSX can we write reactjs code ?

- Yes JSX itself is optional. We can write Reactjs code with out using JSX aswell. But its hard to read and understand.
- Browsers will not be able to understand JSX, so this jsx will be inturn converted into javascript code by build tools like vite/Babel.

Now lets try to write the code in reactjs without using jsx and with using JSX.

- As we have discussed earlier, Entry point in our react application will be main.jsx
- In this main.jsx we are calling App component. Now lets display some heading using JSX and without using JSX.

WithOut JSX:

1. Here iam going to create an h1 element.
 - We need to use React.createElement for this. Which takes 3 parameters.
 - 1st ⇒ element name
 - 2nd ⇒ properties like id, name, className
 - 3rd ⇒ content we need to render.

```
import React, { useState } from 'react'

function App() {
  return React.createElement(
    "h1",
    {id: 'newTag', className: 'newStyle', name: 'newName'},
    "Hello h1 jsx"
  ),
}

export default App
```


2. How can we render nested elements then ?

```
import React, { useState } from 'react'

function App() {
  return React.createElement("div", null,
    React.createElement("h1", null, "Hello h1 jsx"),
    React.createElement("p", null, "paragraph"))
}

export default App
```

With JSX: (Plain React code)

```
import React, { useState } from 'react'

function App() {
  return (
    <div>
      <h1>Hello Iam with JSX</h1>
    </div>
  )
}

export default App
```

Now you will have a question that why this JSX is famous ?

- Because, It is more readable & understandable.
- Code becomes shorter by using jsx.
- Easily maintainable.
- debug is hard

Some JSX rules:

One root element:

1. JSX elements must have only **one root element**.

Not **valid**:

```
function App() {  
  return (  
    <h1>Hello Iam h1 </h1>  
    <h1>Hello Iam another h1</h1>  
  )  
}
```

Valid:

```
function App() {  
  return (  
    <div>  
      <h1>Hello Iam h1 </h1>  
      <h1>Hello Iam another h1</h1>  
    </div>  
  )  
}
```

JavaScript Expressions:

- Javascript expressions are used to display dynamic values in the UI.
- Suppose if we are getting some values from api and we want to display those in the UI we will use expressions.

Example 1:

```
import React, { useState } from 'react'  
  
function App() {  
  const name="Saikrishna";
```

```

    return (
      <div>Hi {name}</div>
    )
  }
  export default App

```

Example 2:

I need to display user information data recieved from the api :

```

import React, { useState } from 'react'

function App() {
  const user={
    name: "Saikrishna",
    company: "Learndepth"
  }

  return (
    <div>Hi {user.name} {user.compaany}</div>
  )
}
export default App

```

Fragments:

- React Fragments will allow us to group multiple elements together without adding extra HTML element into the dom.
- Performance will be slightly better when we use fragments because its not adding extra node into the dom.

```

import React from "react";

function App() {
  return (
    <React.Fragment>
      <h1>H!</h1>
    </React.Fragment>
  )
}

```

```

    <h1>HJ</h1>
  </React.Fragment>
);
}

```

```
export default App;
```

===== Shorter Fragment syntax =====

```
import React from "react";
```

```

function App() {
  return (
    <>
      <h1>H!</h1>
      <h1>HJ</h1>
    </>
  );
}

```

```
export default App;
```

Interview questions & Answers:

1. What is JSX?

Answer:

- JSX stands for **JavaScript XML**.
- It is a **syntax extension of JavaScript** that allows us to write HTML-like code inside JavaScript files. JSX is not HTML. It is converted into JavaScript by tools like **Vite, Babel, or esbuild**.

2. Is JSX mandatory in React?

Answer:

No, JSX is **optional** in React. We can write React code without JSX using `React.createElement()`. However, JSX is preferred because it is **more readable, easier to write, and easier to maintain**.

3. What parameters does `React.createElement()` accept?

Answer:

`React.createElement()` accepts **three parameters**:

1. Element type (e.g., `"div"`, `"h1"`)
2. Props (attributes like `id`, `className`)
3. Children/content to render

4. What is a React Fragment?

Answer:

A React Fragment allows grouping multiple JSX elements **without adding extra HTML elements** to the DOM.

5. What are JavaScript expressions in JSX?

Answer:

JavaScript expressions allow us to display **dynamic data** in JSX using `{}`.

Example:

```
<h1>Hello {name}</h1>
<h2>Total: {price * quantity}</h2>
```

Mini Assignments:

1. Create a product card using below json which we are receiving from api:

Important: Need code with using JSX & without using JSX

Below is the api response for products:

```
const product = {  
  name: "iPhone 15",  
  price: 79999,  
  category: "Mobiles",  
};
```

Expected Result:

iPhone 15

Category: Mobiles

Price: ₹79999

Components:

- Components are building blocks of our react application.

- Each component will represent a part of UI and all components combine together to form a full fledged react application.
- Each component will contain mainly 3 parts.,
 - UI: What user sees is the ui.
 - Logic: What we are doing when user performs some action is called logic. (When button is clicked, When form is submitted)
 - State: Every component will contains its own state and that can change over time (Count, information entered by user, radio , checkbox selected by user etc)

Example:

```
import React, { useState } from "react";
import "./App.css"

function App() {

  const [data, setData] = useState("")

  const handleInput = (data) => {
    console.log(data)
    setData(data)
  }

  return (
    <div>
      <input onChange={(e) => {handleInput(e.target.value)}} />
      <h1>Info: {data}</h1>
    </div>
  );
}

export default App;
```

Types of components:

- There are 2 types of components.,
 1. Class components
 2. Function components

Class Components:

- Before 16.8 version of react, state and life cycle methods are present only in class components.
- But In 16.8 version hooks were introduced in function components to handle state & life cycle events. Now whatever we can do with class components can be achieved by using function components.
- Now Reactjs documentation itself states that we should not use class components for creating components.

Example of component creation using class components:

```
import { Component } from "react";

export class Welcome extends Component {

  render() {
    return <h1>Welcome</h1>;
  }
}
```

- Component:
 - Component is the base class for all our class components
 - It inherits all the features like render(), state, props and life cycle methods. If we are not extending from Component class then we will not have access to these.
- render() :
 - Whenever the component loads, render will be called automatically and the jsx present in render method will be displayed in the UI.

Function components:

- Function components are new way to create components in reactjs.

- In 16.8 version of reactjs, Hooks were introduced. Now whatever we can do with class components can be almost achieved using function components.

Example of component creation using function components:

```
function Welcome() {  
  return <h1>Welcome</h1>;  
}  
  
export default Welcome;
```

- As per standards, If we want to export these components in other files, use default export.
- Component name should be in Capital Case (Each word will be starting with Capital Letter)
 - Eg: UserProfile, Header etc

Nested Components:

In reactjs we can use one component inside another component. These are called as nested components.

- The outer component is called parent component & the inner component is called child component.
- Lets say i have to display button in multiple pages, then i will create separate component for button and i will import it in various components.
- By using nested components we can .,
 - Reuse UI & Avoid code duplication
 - complexity will be reduced

```
function Input() {  
  return <input placeholder="Enter name" />;  
}  
  
function Form() {
```

```
return (  
  <div>  
    <Input />  
    <button>Submit</button>  
  </div>  
);  
}
```

Interview questions & Answers:

1. What is a component in React?

Answer:

A component is a **building block of a React application**.

Each component represents a **part of the UI**, and multiple components together form a complete React application.

2. What are the types of components in React?

Answer:

There are two types of components:

1. **Class components**
2. **Function components**

3. What is a class component?

Answer:

A class component is a JavaScript class that:

- Extends `React.Component`
 - Uses `render()` to return JSX
-

4. Why was `Component` class extended in class components?

Answer:

By extending `Component`, the class gets access to:

- `render()` method
- `this.state`
- `this.props`
- Lifecycle methods

Without extending `Component`, these features are not available.

5. What are function components?

Answer:

Function components are simple JavaScript functions that:

- Return JSX
 - Represent UI
 - Use Hooks to manage state and lifecycle logic
-

6. What are Hooks and why were they introduced?

Answer:

Hooks were introduced in React 16.8 to:

- Add state and lifecycle features to function components
 - Avoid class components
 - Reuse logic easily
 - Write cleaner and simpler code
-

7. What are nested components? Why they are used ?

Answer:

Using one component inside another component is called **nested components**.

Nested components help to:

- Reuse UI
 - Avoid code duplication
 - Improve maintainability
 - Build scalable applications
-

Mini Assignment:

1. Create the following components:

- `Home` (Parent Component)
 - `Header` (Reusable Component)
 - `ProductList` (Reusable Component)
 - `Footer` (Reusable Component)
- The `Home` component should:
 - Import `Header` , `ProductList` , and `Footer`
 - Render all three components inside it

- Each component should return JSX content
 - Header → Food Delivery App
 - ProductList → List of Popular items(Burger, Pizza)
 - Footer → ©2026 Food delivery

Expected UI Result:

Food Delivery App

Popular Items

Burger

Pizza

© 2026 Food Delivery

Event Handling:

In this session we will cover following topics.

1. Synthetic events
2. Click events
3. Common mistakes in event handling
4. Input events
5. Inline event handling
6. Common mistakes in inline event handling

2 Mini Assignments , 1 interview question & answer

Synthetic events:

- In older browsers, there were lot of inconsistencies related to event objects.
- In chrome, developers used to access the event object using `event.target` where as in older Internet explorer they used to access event object using `event.srcElement`
- Similarly for listening to click events, developers used to write different code for different browsers.
- To resolve all these inconsistencies they used jquery and jquery has inbuilt mechanism to handle browser inconsistencies.
- Similarly, React has its own way to handle these inconsistencies using Synthetic events. It will create a wrapper around browser inbuilt events and provides a unified way for event handling.

EventHandlers:

Now lets see how to write event handlers in JSX.

- In jsx we need to use **camel case** for Event handlers(In Html , its lowercase). Eg: **onClick, onChange**

```
import React, { useState } from 'react'
import './new.css';

function App() {

  function handleClick() {
    alert("Button clicked!");
  }

  return (
    <button onClick={handleClick}>Click</button>;
  )
}
export default App
```

Common mistake developers make:

Incorrect format:

```
<button onClick={handleClick()}>Click</button>;
```

```
/* Here you are calling the function, so this will be called immediately when the component is rendered */
```

Correct format:

```
<button onClick={handleClick}>Click</button>;
```

```
// Here we are passing function, so it will be called only when user clicks on the button
```

Passing arguments to event handlers

Sometimes you need **data in the handler**.

```
function handleClick(name) {  
  alert(`Hello ${name}`);  
}  
  
function App() {  
  return (  
    <button onClick={() => handleClick("Sai")}>  
      Greet  
    </button>  
  );  
}
```

How to handle Input events :

Lets learn how we can handle input change in jsx.

- Here Reactjs will pass event object automatically when event occurred.

```
function App() {
  const handleInputChange = (e) => {
    console.log(e.target.value);
  };

  return (
    <>
      <input onChange={handleInputChange} />
    </>
  );
}

export default App;
```

Inline Event Handler:

You can also define the function **inline** using an arrow function:

```
function App() {
  return (
    <button onClick={() => alert("Button clicked!")}>
      Click Me
    </button>
  );
}
```

Common mistakes in Inline event handling:

Incorrect format:

```
<button onClick={alert('You clicked me!')}>
```

/* Here this code inside onclick will be executed when component is rendered. Because we are directly writing the code instead of calling a function


```
*/
```

Correct Format:

```
<button onClick={() => alert('You clicked me!')}>
```

Interview Questions & Answers:

1. What are Synthetic events ?

- Synthetic events are the wrapper around native browser events.
- By using this react will remove browser inconsistencies and provides a unified api interface for event handling

Mini Assignments:

1. Handle dropdown events:

When user selects dropdown value, it should be displayed in the console.

2. Implement checkbox event handling using a single event handler function and log the selected values to the console

Rendering List & Keys

In this session we will learn.,

- How to render a list.
- Iterate over array of objects
- Iterate over array of arrays
- Filtering the Items

- Adding click event on array of Items
- Keys in react & examples

2 Interview questions & Answers

1 Mini assignment as home work

Rendering List:

- Now lets see how we can display list of array items in UI. We will use Map() method to render a list.
 - Map method will iterate over each element of the array and each iteration will return JSX
- Lets render a simple list which contains fruits names.

```
function App() {  
  const fruits = ["Apple", "Orange", "Banana", "Grape  
s"];  
  return (  
    <>  
      {fruits.map((fruit) => {  
        return <h1>{fruit}</h1>;  
      })}  
    </>  
  );  
}  
  
export default App;
```

Common mistake:

- Common mistake developers will do in the interviews here is, they **will not return the jsx** in each iteration.
-

Iterate over array of objects:

In real time projects, you will not display this plain arrays. You will be working with array of objects.

Examples:

- Product List, category list etc.

Here i have some menu list which contains vegetables & fruits

```
function App() {
  const menuList = [
    { id: 1, name: "Vegetables" },
    { id: 2, name: "Fruits" },
  ];

  return (
    <ul>
      {menuList.map((menu) => {
        return (
          <h1>
            {menu.id} {menu.name}
          </h1>
        );
      })}
    </ul>
  );
}

export default App;
```

Iterate over Array of Arrays:

Now we will see how we can iterate and display array of arrays.

```
function App() {
  const menuList = [
    { id: 1, name: "Vegetables", items: ["Onion", "Carrot", "Beans"] },
    { id: 2, name: "Fruits", items: ["Apple", "Orange"] },
  ];

  return (
```

```

    <ul>
      {menuList.map((menu) => {
        return (
          <>
            <h1>{menu.name}</h1>
            <div>{
              menu.items.map((item) => {
                return <h3>{item}</h3>
              })
            }</div>
          </>
        );
      })}
    </ul>
  );
}

export default App;

```

Filtering the Items:

Now i want to display only vegetables. For this we can use filter method.

Filter:

Filter will loop through each element of the array and returns a new array with only those elements which satisfies specific condition.

```

function App() {
  const menuList = [
    { id: 1, name: "Vegetables", items: ["Onion", "Carrot", "Beans"] },
    { id: 2, name: "Fruits", items: ["Apple", "Orange"] },
  ];

  return (
    <ul>

```

```

    {menuList
      .filter((list) => list.name === "Vegetables")
      .map((menu) => {
        return (
          <>
            <h1>{menu.name}</h1>
          </>
        );
      })}
    </ul>
  );
}

export default App;

```

Mistakes:

While comparing two values, Most of the developers use Loose equality (==) operator but you should always use strict equality (===) operator for checking both type aswell as data.

Adding click event on item list:

- Now lets say i want to display buttons with item names.
- When user clicks on that button, it should log the item name into the console.

```

function App() {
  const menuList = ["Apple", "Orange"];

  const handleItemClick = (listItem) => {
    console.log(listItem, "Iam logged");
  };

  return (
    <>
      {menuList.map((listItem) => {

```

```

        return (
            <>
                <button
                    onClick={() => {
                        handleItemClick(listItem);
                    }}
                >
                    {listItem}
                </button>
            </>
        );
    }
}

export default App;

```

Keys in JSX:

- In Reactjs, Keys are used internally to uniquely identify which elements have been added, updated or removed during from the dom the reconciliation process.

```

import React from "react";

function App() {
    const products = [
        { id: 1, name: "Phone" },
        { id: 2, name: "Laptop" },
        { id: 3, name: "Headphones" }
    ];

    return (
        <div>
            <h2>Product List</h2>

```

```

    <ul>
      {products.map(product => (
        <li key={product.id}>
          {product.name}
        </li>
      ))}
    </ul>
  </div>
);
}

export default App;

```

Keys should be unique:

Keys should be always unique else it will throw an error in the console.

Eg: I will create objects with same id as 1.

```

const menuList = [
  { id: 1, name: "Vegetables", items: ["Onion", "Carrot", "Beans"] },
  { id: 1, name: "Fruits", items: ["Apple", "Orange"] },
];

```

✖ ▶ Encountered two children with the same key, `1`. Keys should be unique so [installHook.js:1](#) that components maintain their identity across updates. Non-unique keys may cause children to be duplicated and/or omitted – the behavior is unsupported and could change in a future version.

What are the best choices for keys ?

- Id's from database / backend api.
- If we dont get it from api then generate uuid for each object in UI.

Bad choices :

- Indexes of array. (When ever we reorder list/inserting or deleting element in list it will break the order).

Interview Questions & Answers:

1. What are keys in React ?

- Keys are used to uniquely identify the elements in the list.
- react will use this to identify, which elements in the list have been added, removed or updated.

```
function MyComponent() {  
  const items = [  
    { id: 1, name: "apple" },  
    { id: 2, name: "banana" },  
    { id: 3, name: "orange" }  
  ];  
  
  return (  
    <ul>  
      {items.map((item) => (  
        <li key={item.id}>{item.name}</li>  
      ))}  
    </ul>  
  );  
}
```

2. What are the best choices for keys?

- Database IDs
 - Backend API IDs
 - UUIDs (if ID is not provided)
 - We should not use Array indexes as keys.
-

Mini Assignments:

Assignment 1:

Problem Statement

You are building a **Food Menu UI** for a restaurant application.

You are given the following data:

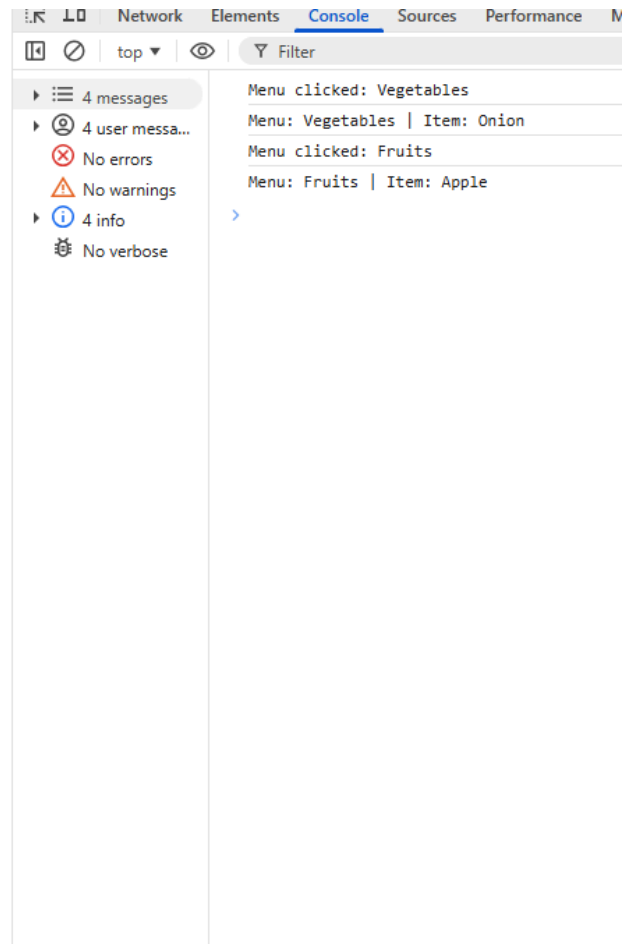
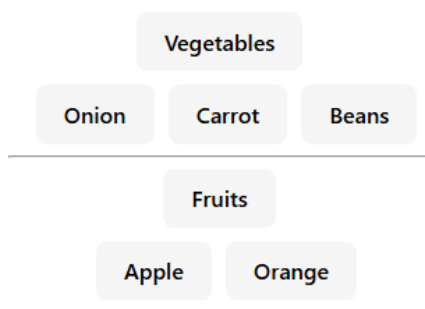
```
const menuList = [
  {id:1,name:"Vegetables",items: ["Onion","Carrot","Beans"]
},
  {id:2,name:"Fruits",items: ["Apple","Orange"] },
];
```

Requirements

1. Display **menu categories** (`Vegetables` , `Fruits`) as **buttons**.
2. When a **menu button** is clicked:
 - Log the **menu name** in the console.
3. Under each menu, display **item buttons** (`Onion` , `Carrot` , etc.).
4. When an **item button** is clicked:
 - Log **both menu name and item name**.
5. Use **nested** `map()` properly.
6. Use **unique keys** at both levels.
7. Do **not** use `forEach()` for rendering.
8. Avoid unnecessary wrapper `div` s (use fragments where needed).

Expected Output:

Focus only on console logs (`Styling is not yet covered so Ignore styling part`)



Conditional Styling:

In this session we are going to learn.,

1. Logical And Operator
2. Ternary Operator
3. Conditionally displaying components

2 Mini Assignments, 1 Interview question and answer

- If we want to show different UI based on the condition then we will use conditional rendering.
- We will not be able to use if else conditions inside JSX. So we will be using ternary operator and Logical AND operator.

Logical And (&&):

Lets say i want to display "Welcome Admin" if user is admin else i need to show "Welcome User" message.

```
function App() {  
  const isAdmin = true;  
  
  return (  
    <div>  
      {isAdmin && <h1>Welcome Admin</h1>}  
      {!isAdmin && <h1>Welcome User</h1>}  
    </div>  
  );  
}  
  
export default App;
```

Ternary operator :

- Now how can we achieve the same in single line ? ie., by using ternary operator.
- Ternary operator is same like if else condition.

```
import React, { useState } from 'react'  
  
function App() {  
  const isLoggedIn = false;  
  
  return <>  
    {isLoggedIn ? <h1>Hi Sai</h1> : <h1>Please login</h1>}  
  </>  
}  
  
export default App
```

Conditional Rendering components:

I want to display header and footer components dynamically based on some flag.

```
Header:
function Header() {
  return <h1>Food Delivery App</h1>;
}

export default Header;

Footer:
function Footer() {
  return <p>© 2026 Food Delivery</p>;
}

export default Footer;

App:
function App() {
  const isHeaderEnabled = false;

  return <div>{isHeaderEnabled ? <Header></Header> : <Foote
r></Footer>}</div>;
}

export default App;
```

Mini Assignments:

1. Problem Statement

You are working on an **E-commerce Product Listing Page**.

The product data is fetched from an API and stored in a `products` array.

Your task is to **conditionally render the UI** based on the following scenarios:

1. **If the API returns an empty product list**, Display ⇒ `No products available`
2. **If products are available**, display the product names in a list.

Below is the sample data:

Case 1: Products are empty:

```
const products = [];
```

Case 2: Products array contains some data:

```
const products = [  
  {id:1,name:"iPhone 15" },  
  {id:2,name:"Samsung S24" }  
];
```

✓ Requirements

- Use **conditional rendering**
- Handle **empty list case**
- Use `.map()` correctly
- Each product must have a **unique key**
- Code should be **clean and readable**

Expected UI Results:

If product list is empty

No products available

If product list contains some data.

Cycle Carroms

Solution:

You need to solve the problem on your own

2. Problem Statement

You are developing an **E-commerce Product Listing Page**.

Products are fetched from an API and stored in some variable(Take static data for this as we have not discussed about api calls) you need to **render different UI messages using nested conditional rendering**.

Scenarios to Handle

1. If data is not available

👉 Show: `No products available`

2. If products are available

- If user is logged in

👉 Show product list

- If user is NOT logged in

👉 Show: `Please login to view products`

Given Data

(Use Static data `as` we have not learned about api calls)

```
const isLoading = false;
const isLoggedIn = true;

const products = [
  {id:1, name: "MacBook Pro" },
  {id:2, name: "iPhone 15" }
];
```

Requirements

- Use **nested conditional rendering**
- Handle **loading, empty list, and authentication**
- Use `.map()` for rendering products
- Add **unique key** to each item
- Code must be **clean and readable**

Solution:

You need to solve the problem on your own

Interview questions & Answers:

1. What is conditional rendering in React?

Answer:

Conditional rendering in React means **displaying different UI components or elements based on some condition** (true or false).

React uses **JavaScript expressions**, so conditions are handled using:

- Ternary operator (`? :`)
- Logical AND (`&&`)
- `if-else` outside JSX

Styling:

In this session we will learn.,

- Inline styling
- Inline styling with variables
- Dynamic Inline styling based on some condition
- Css Stylesheets
- Css Modules

3 Interview Ques and Ans

1 Mini Assignment

Inline styling:

- Here, We will use style attribute to apply inline styles to an element.

```
<div  
  style={{
```



```

    backgroundColor: "lightblue",
    padding: "20px",
    fontSize: "18px"
  }}
>
  Inline Styled Div
</div>

```

Important rules

- Style value must be an object.

```
style={{ color: "red" }}
```

- CSS properties should use **camelCase**

```

background-color ✗
backgroundColor ✓

```

```

font-size ✗
fontSize ✓

```

- Values are strings usually, If we provide numbers then it will be converted to pixels by default.

```

fontSize: "16px"

fontSize: 16 // This will be converted into px by default.

```

Inline styles with variable:

Lets say i have created another div with "Hello Teachers" string and i want to have same styles for this new div as well.

```

function App() {

  return (
    <>
      <div
        style={{
          backgroundColor: "yellow",
          fontSize: 100,
          color: "blue",
        }}
      >
        Hello Students
      </div>
      <div
        style={{
          backgroundColor: "yellow",
          fontSize: 100,
          color: "blue",
        }}
      >
        Hello Teachers
      </div>
    </>
  );
}

export default App;

```

- Now in this case if we want to reuse these same styles in multiple places, we will create style in a variable and we will reuse it anywhere we want.

```

function App() {
  const boxStyle = {
    backgroundColor: "yellow",
    fontSize: 100,
    color: "blue",
  };

```

```

return (
  <>
    <div style={boxStyle}>Hello Students</div>
    <div style={boxStyle}>Hello Teachers</div>
  </>
);
}

export default App;

```

Dynamic Inline styling based on condition

- If we want to apply inline style based on some condition, then we will use dynamic inline styling.

```

const isLoggedIn = true;

return (
  <>
    <h1 style={{ backgroundColor: isLoggedIn ? "green" :
"red" }}>
      Hello Students
    </h1>
  </>
);

```

Dynamic Inline style with variable based on condition:

- Lets say i want to apply inline style with variable based on some condition.

```

function App() {
  const boxStyle1 = {

```

```

    backgroundColor: "yellow",
    fontSize: 100,
    color: "blue",
  };

  const boxStyle2 = {
    backgroundColor: "red",
    fontSize: 100,
    color: "blue",
  };

  const isLoggedIn = false;

  return (
    <>
      <h1 style={isLoggedIn ? boxStyle1 : boxStyle2}>
        Hello Students
      </h1>
    </>
  );
}

export default App;

```

Common mistakes developers make:

```

// ❌ Wrong
style="color: red" // This is html way

// ❌ Wrong
style={{ background-color: "red" }}

// ❌ Wrong
style={{ fontSize: 16px }}

```

CSS Stylesheet:

- Here we will write our styles in separate .css file and we will use it in our component by using **className** attribute.

Parent.jsx

```
import './Parent.css';

const Parent = () => {
  return <h1 className="boxStyle">Parent</h1>;
};

export default Parent;
```

Parent.css:

```
.boxStyle {
  background-color: green;
  color: white;
  padding: 40px;
}
```

Css Styles based on condition:

Lets say you want to conditionally apply css styles.,

Parent.jsx:

```
import './Parent.css';

const Parent = () => {
  const isLoggedIn = false;
  return <h1 className={isLoggedIn ? "boxStyle1" : "boxStyle2"}>Parent</h1>;
};
```

```
export default Parent;
```

```
Parent.css:
```

```
.boxStyle1 {  
  background-color: green;  
  color: white;  
  padding: 40px;  
}
```

```
.boxStyle2 {  
  background-color: red;  
  color: white;  
  padding: 40px;  
}
```

CSS Modules:

Lets say i have Products & Orders components and css files related to those components as shown below

```
src/  
├─ components/  
│   ├─ Products.jsx  
│   ├─ Products.css  
│   ├─ Orders.jsx  
│   └─ Orders.css
```

- Now lets say i have same style in both Orders.css and Products.css files as shown below

Products.jsx:

```
import "../Products.css";

const Products = () => {
  return <h1 className="hStyle">Parent</h1>;
};

export default Products ;
```

```
=====
=====
```

Products.css:

```
.hStyle{
  background-color: blue;
}
```

```
=====
=====
```

Orders.jsx:

```
import "../Orders.css"

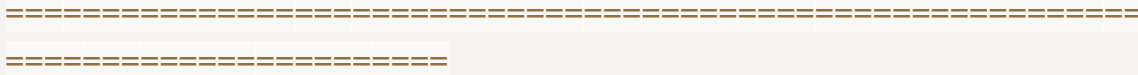
const Orders= () => {
  return <h1 className="hStyle">Orders</h1>;
};

export default Orders;
```

```
=====
=====
```

Orders.css:

```
.hStyle{
  background-color: red;
}
```



App.jsx:

```
import Products from "../Components/Products";
import Orders from "../Components/Orders ";

function App() {
  return (
    <>
      <Orders></Orders>
      <Products></Products>
    </>
  );
}

export default App;
```

- If i open browser, it will apply same background colour ie., Red from orders.css. But As per our expectation it should be blue in Products and Red in Orders. So why it applying same colour here ?
- Thats because all these css will be combined internally into single css during runtime.
- So which ever css loads last ie., in imports of app component, orders component is imported last so orders css will be loaded last and it will be applied to both elements.
- To resolve these styling conflict issues, we will use css modules in react.

Css Modules will solve this problem by making styles local and converts each class name into unique name internally.

- `.hStyle` in Products → `Products_hStyle__x7a9d`
- `.hStyle` in Orders → `Orders_hStyle__x9y8`

Products.jsx:

```
import productsStyle from "../Products.module.css";

const Products = () => {
  return <h1 className={productsStyle.hStyle}>Parent</h1>;
};
```

```
export default Products;
```

```
=====
=====
```

Products.module.css:

```
.hStyle{
  background-color: blue;
}
```

```
=====
=====
```

Orders.jsx:

```
import ordersStyle from "../Orders.module.css"
```

```
const Orders= () => {
  return <h1 className={ordersStyle.hStyle}>Orders</h1>;
};
```

```
export default Orders;
```

```
=====
=====
```

Orders.module.css:

```
.hStyle{
  background-color: red;
}
```

=====

App.jsx:

```
import Products from "../Components/Products";
import Orders from "../Components/Orders";

function App() {
  return (
    <>
      <Orders></Orders>
      <Products></Products>
    </>
  );
}

export default App;
```

Styled Components:

- By using styled components, we can write css inside js file.
 - It works same like Css Modules.
 - You can learn later if needed.
-

Mini Assignments:

1. Product List with Inline Style Variables

Scenario:

Render a list of products in a grid. Use **inline style variables** to reuse styling for all cards.

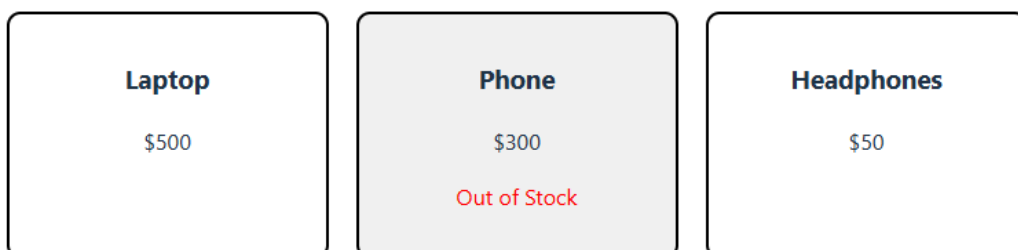
Requirements:

1. Create `ProductList` component. Products data:

```
const products = [  
  {name: "Laptop", price: 500, isInStock: true },  
  {name: "Phone", price: 300, isInStock: false },  
  {name: "Headphones", price: 50, isInStock: true }  
];
```

1. Define `cardStyle` variable to reuse inline styles.
2. Grid layout: **display cards in row** (use `display: flex` on container).
3. Out of Stock products should have **light gray background**.

Expected UI Output:



Solution:

Try your own & come with solution tomorrow

Interview questions & Answers:

1. What are CSS Modules in React?

Ans:

- CSS Modules make styles **local to the component**.
- Class names are **converted to unique names internally**, e.g., `.hStyle` → `Products_hStyle__x7a9d`.
- Prevents style conflicts across components.

```
import styles from './Products.module.css';  
<h1 className={styles.hStyle}>Product</h1>
```

2. What are inline styles in React? How do they differ from HTML inline styles?

Ans:

- Inline styles in React are applied using the `style` attribute, and the value must be a **JavaScript object**.
- CSS properties are written in **camelCase**, not kebab-case.
- Values are usually strings; numbers for units like `px` are automatically converted.

```
<div style={{backgroundColor:"blue",fontSize:16 }}>Hello</div>
```

Difference from HTML:

- HTML: `style="background-color: blue;"`
- React: `style={{ backgroundColor: "blue" }}`

3. What happens if two CSS files have the same class name?

Ans:

- All CSS files are combined internally.
 - The **last loaded CSS** will override the previous styles.
 - This can cause **style conflicts**, which is solved by **CSS Modules**.
-

State:

In this session we are going to cover,

- State in reactjs
- State in function components
- Exp 1: Usestate with string - Display data entered in Textbox in UI
- Exp2: Usestate with number - Implement increment & decrement counter
- Exp3: Usestate with Object - Implement Signup form
- Exp4: Usestate with Arrays - Todo list Implementation

4 mini assignments: Most used implementations from real time projects & Most asked in interviews

- **State** is used to store **data of a component**.
- This data can **change over time**.
- When **state changes**, React **re-renders the component** and updates the UI automatically.

Real-time examples

- Signup / Login form → store user input in state
- API response → store fetched data in state
- Counter value → store count in state
- Todo list items → store list in state

State in Functional Components

In functional components, we use a **hook** called `useState`.

Syntax of `useState`

```
const [stateVariable, setStateFunction] = useState(initialValue);
```

Breakdown:

Part	Meaning
<code>stateVariable</code>	Current state value
<code>setStateFunction</code>	Function to update state
<code>initialValue</code>	Initial value of state

Example 1: Textbox Input & Display (String in state)

- Lets say user has entered data in text box and we want to display that in UI

```
function App() {  
  
  const [userInfo, setUserInfo] = useState("");  
  
  const handleUserInput = (e) => {  
    console.log(e.target.value)  
    setUserInfo(e.target.value)  
  }  
  
  return <>  
    <input style={{height: 30}} onChange={handleUserInput}/>  
    <h1>{userInfo}</h1>  
  </>  
}  
  
export default App;
```

What is happening here?

1. User types in input box
2. `onChange` event triggers
3. `setUserInfo` updates state
4. Component re-renders
5. Updated value appears in `<h1>`

State Type	Correct Update Pattern
Number	<code>setCount(prev => prev + 1)</code>
String	<code>setName(prev => prev + "!")</code>
Object	<code>setUser(prev => ({ ...prev, key: value })))</code>
Array	<code>setList(prev => [...prev, item])</code>

Example 2: Counter (Increment & Decrement) (Numeric value in state)

Implement a counter component with Increment & decrement buttons

Here we will create a counter component which contains increment & decrement buttons to change the count

```
function App() {  
  const [count, setCount] = useState(0);  
  
  return (  
    <>  
      <button  
        onClick={() => {  
          setCount((prevCount) => prevCount - 1);  
        }}  
      >  
        -  
      </button>  
      <h1>{count}</h1>  
      <button
```

```

      onClick={() => {
        setCount((prevCount) => prevCount + 1);
      }}
    >
      +
    </button>
  </>
);
}

export default App;

```

Example 3: useState with Objects (Signup Form) (Storing object in state)

- Lets say i have a signup form where user enters his data.

```

import { useState } from "react";

const Parent = () => {
  const [user, setUser] = useState({
    name: "",
    email: "",
    city: "",
  });

  return (
    <>
      <input
        placeholder="Name"
        onChange={(e) => setUser({ ...user, name: e.target.
value })}
      />

      <input
        placeholder="Email"
        onChange={(e) => setUser({ ...user, email: e.targe

```



```

t.value }}}
    />

    <input
      placeholder="City"
      onChange={(e) => setUser({ ...user, city: e.target.
value })}}
    />

    <h1>{user.name}</h1>
    <h1>{user.email}</h1>
    <h1>{user.city}</h1>
  </>
);
};

```

```
export default Parent;
```

```

===== Generic (Industry Standard) way
=====

```

```

import { useState } from "react";

function Parent() {
  const [user, setUser] = useState({
    name: "",
    email: "",
    city: "",
  });

  const handleChange = (e) => {
    setUser({
      ...user,
      [e.target.name]: e.target.value,
    });
  };
}

```

```

return (
  <>
    <input name="name" placeholder="Name" onChange={handleChange} />
    <input name="email" placeholder="Email" onChange={handleChange} />
    <input name="city" placeholder="City" onChange={handleChange} />

    <h1>{user.name}</h1>
    <h1>{user.email}</h1>
    <h1>{user.city}</h1>
  </>
);
}

export default Parent;

```

How spread operator works:

```

const oldObj = {
  name: "sai",
  email: "ssss@gmail.com",
};

const obj = {
  ...oldObj,
  name: "kriddshna"
}

console.log(obj)

```

Explanation (Important)

- `...oldObj` copies all existing properties
- `name: "krishna"` overrides old `name`
- Original object **will not change**

👉 Always use spread operator while updating objects in state

Example4:

useState with Arrays: Todo List

We are going to create a **Todo application** with the following features:

- We will have **one textbox** where the user can type a todo item.
- We will have an **"Add Todo" button**.
- When the user enters some text and clicks on **Add Todo**:
 - The todo item should be **added to the todo list**
 - The **textbox should be cleared** automatically

Improvement 1:

- Each todo item should have a **Delete button**.
- When the user clicks on **Delete**:
 - That particular todo item should be **removed from the list immediately**

Improvement 2:

- If the textbox is **empty**, the todo **should not be added**.
- If the same todo item already exists in the list:
 - It **should not be added again** (no duplicates allowed)

Improvement 3:

- Disable Add todo button if user has not entered any data.

```
import React, { useState } from "react";
import "./App.css";

function App() {
  const [data, setData] = useState("");
  const [todoList, setTodoList] = useState([]);

  const handleClick = () => {
```

```

    setTodoList([...todoList, data]);
    setData("");
  };

  const handleDelete = (listItem) => {
    let updatedTodos = todoList.filter((item) => item !== l
istItem);
    setTodoList(updatedTodos);
  };

  return (
    <>
      <input
        onChange={(e) => {
          setData(e.target.value);
        }}
        value={data}
      />
      <button onClick={handleClick} disabled={!data}>
        Add Todo
      </button>
      <ol>
        {todoList.map((listItem) => {
          return (
            <li>
              {listItem}
              <button
                onClick={() => {
                  handleDelete(listItem);
                }}
              >
                Delete
              </button>
            </li>
          );
        })}
      </ol>
    </>
  );

```

```
);  
}  
  
export default App;
```

Mini Assignments:

(All these are REAL time project implementations)

Make sure you complete this before tomorrow's class. I will randomly choose a student to walk us through the solution.

This exercise is meant to strengthen your ReactJS concepts and prepare you for real-world job interviews.

1. Implement Dark mode and Light mode:

We will implement a **Dark and Light mode switch** in a React application using `useState`, conditional rendering, and conditional styling.

What you will learn:

After completing this assignment, you will learn:

- Use boolean state to control UI behavior
- Apply conditional styles based on state
- Understand how state changes trigger re-rendering
- Implement real-world UI features using React basics

Requirement:

- Use a **boolean state variable** to manage theme and default theme must be **Light mode**. Update state **only using the setter function**.
- Display a **toggle button** on the UI and On button click, Switch between **Dark** and **Light** mode.
 - Update button text dynamically:

- Light mode → *"Switch to Dark Mode"* and in Dark mode → *"Switch to Light Mode"*
- Change **background color** based on theme:
 - Light → #ffffff
 - Dark → #121212
- Change **text color** for readability:
 - Light → Black
 - Dark → White
- Show a **heading** indicating current mode *Light Mode / Dark Mode*

Expected UI:

Light Mode

Switch to Dark Mode

Dark Mode

Switch to Light Mode

2. Dynamic States Based on Country (Used in most of ecommerce signup forms)

Implement a **dependent dropdown** in React where **selecting a country dynamically loads the states** of that country using React state.

Concepts Covered

- `useState` hook
- Event handling (`onChange`)
- Conditional rendering
- Dynamic lists (arrays in state)
- Mapping arrays to `<option>` in `<select>`

Requirements

1. Country Dropdown

- Provide a dropdown with at least **3 countries** (e.g., USA, India, Australia).
- Selecting a country updates the **state list** dynamically.

2. State Dropdown

- Display states/provinces corresponding to the selected country.
- Initially, the state dropdown should **show "Select country first"**.

3. State Management

- Use `useState` for:
 - Selected country
 - Selected state
 - States list

4. Dynamic Rendering

- State dropdown updates **immediately** when a country is selected.
- If no country is selected, show a placeholder in the state dropdown.

5. Display

- Show the **selected country and state** below the dropdowns.

Expected UI:

Country & State Selector

Country:

State:

Country & State Selector

Country:

State:

Selected: India - Maharashtra

3. Implement star rating component:

We will implement a **product rating component** where a user can select a rating (1–5 stars) and the UI updates dynamically to reflect the selected rating.

Requirements

1. Rating System

- Implement a **5-star rating** system for a product.
- Users should be able to **click on a star to set the rating**.

2. State Management

- Use `useState` to store the **selected rating**.
- Default rating should be **0**.

3. Dynamic Highlight

- Selected stars should be highlighted (different color, e.g., gold).
- Unselected stars should appear inactive (e.g., gray).

4. Display Selected Rating

- Below the stars, display the **numeric rating** dynamically.

Expected UI:

Product Rating



Selected Rating: 4 / 5

4. Implement Accordion Functionality Using React State

Create an **accordion component** where clicking a section heading expands or collapses the corresponding content dynamically using React state.

Requirements

1. Accordion Structure

- Create **3–4 sections**.
- Each section must have:
 - A **heading/title**
 - **Hidden content** that expands on click

2. State Management

- Use `useState` to track **currently active/expanded section**.
- Initially, all sections should be **collapsed**.

3. Expand/Collapse Behavior

- Clicking a section heading toggles its content visibility.
- Only one section can be expanded at a time (optional: multiple can expand if you want advanced).

4. Dynamic Rendering

- Expanded section shows its content.
- Collapsed sections hide content.

5. Styling

- Section headers should look clickable (pointer cursor).
- Highlight the active section (optional: background color change).

Expected UI:

Accordion Component

Section 1

Section 2

This is the content of Section 2

Section 3

Interview Questions & Answers

1. What is state in React?

- State is a built-in object that stores component's data which can change over time.
- When state changes, the component re-renders and refreshes the UI.

2. Why do we need state in React?

To manage dynamic data like form inputs, toggles, counters, and UI interactions without reloading the page.

3. How do you create state in a functional component?

Using the `useState` hook.

```
const [count, setCount] =useState(0);
```

4. How do you update an object in state?

```
setUser(prev => ({ ...prev, name:"Sai" }));
```

5. How do you update an array in state?

```
setTodos(prev => [...prev, newTodo]);
```

Props:

In this session, we will learn.,

- About Props, Syntax & Destructuring
- Sharing Number, Strings, Boolean, Objects, Arrays
- Sending functions as props
- Sending JSX as props using Children props
- State vs Props

5 Interview questions & answers

2 Mini Assignments

-
- Props are one of the way to share the data between the components.
 - If we want to send the data from parent component to child component then we will use props.
 - Eg: If we want to send data from products to products List page.

- these props are immutable(means readonly). We cannot change props in child component.
 - Props are passed as attributes while calling the child components and child components will receive all these props as object.
-

Syntax explanation:

- Create basic parent and child components (child inside parent) and display in app component.
- Props object:
 - Send user name, age and mobile to child from parent

```
import Child from "./Child";

const Parent = () => {
  return (
    <div>
      <h1>Parent</h1>
      <Child name={"saikrishna"} age={28} contact={"9887
877788"}></Child>
    </div>
  );
};

export default Parent;

=====

const Child = (props) => {
  return <h1> Hi {props.name} Age: {props.age} contac
t:{props.contact}</h1>
}

export default Child;
```

Destructuring syntax:

- Destructuring allows us to extract values from arrays or objects and store them in new variables.

```
import Child from "./Child";

const Parent = () => {
  return (
    <div>
      <h1>Parent</h1>
      <Child name={"saikrishna"} age={28} contact={"9887877788"}></Child>
    </div>
  );
};

export default Parent;
```

```
=====
=====

const Child = ( {name, age, contact}) => {
  return <h1> Hi {name} Age: {age} contact:{contact}</h1>
}

export default Child;
```

```
===== (OR) =====
=====

const Child = (props) => {
  const {name, age, contact} = props;
  return <h1> Hi {name} Age: {age} contact:{contact}</h1>
}
```

```
export default Child;
```

Sending Objects:

- Create a userinfo object with properties as name, age and mobile and use them in child component.

```
import Child from "./Child";

const Parent = () => {
  const user = {
    name: "sai",
    age: 28
  };

  return (
    <div>
      <h1>Parent</h1>
      <Child userInfo={user} skills={user.skills}></Child>
    </div>
  );
};

export default Parent;
```

```
===== Child component =====
=====
```

```
const Child = ({userInfo}) => {
  return <h1>{userInfo.name} {userInfo.age}</h1>
}
```

```
export default Child;
```

```
===== Arrays =====  
=====
```

```
const Child = ({ userInfo, skills }) => {  
  return (  
    <>  
      <div>  
        {skills.map((skill) => {  
          return <h1>{skill}</h1>;  
        })}  
      </div>  
    </>  
  );  
};  
  
export default Child;
```

Sending Arrays to child component:

```
import Child from './Child';  
  
const Parent = () => {  
  const skills = ["Javascript", "Reactjs"]  
  
  return (  
    <>  
      <h1>Parent</h1>  
      <Child skills={skills}></Child>  
    </>  
  );  
};
```



```

export default Parent;

===== Child component =====
=====

const Child = ({ skills }) => {
  return (
    <>
      {skills.map((skill) => {
        return <h1>{skill}</h1>;
      })}
    </>
  );
};

export default Child;

```

Default props:

- Default props are the fallback values used by the component when prop is not passed from the parent component.
- Some times we will forget to send the prop value from parent to child component. At that time this default value will be displayed.

```

const Parent = () => {
  return (
    <>
      <Child name="Add to cart"></Child>
    </>
  );
};

```

```
export default Parent;
```

```
=====
```

```
const Child = ({ name="Click me" }) => {  
  return (  
    <>  
      <button>{name}</button>  
    </>  
  );  
};
```

```
export default Child;
```

Functions as props :

- This is one of the most common use case in enterprise applications.
- Lets say i have a button component which is used in cart screen and products screen
 - Button functionality will be different in these 2 screens. So we will create the functions in parent components and send it to child component.

```
import Button from "./Button";
```

```
const Product = () => {  
  const handleClick = () => {  
    console.log("Added to wishlist");  
  };  
  
  return <Button name="Wishlist" handleClick={handleClick}/>
```

```

tClick}></Button>
};

export default Product;

=====

import Button from "./Button"

const AddToCart = () => {

  const handleClick = () => {
    console.log("Added to cart")
  }

  return <Button name="Add to Cart" handleClick={handleClick}></Button>
}

export default AddToCart;

===== Child component Button =====

const Button = ({ name, handleClick }) => {
  return <button onClick={handleClick}>{name}</button>;
};

export default Button;

```

JSX as props: (Very Important in realtime projects)

- If we want to send JSX as the prop, then we will use this children props.
- Children props will allows the component to display dynamic content passed to that component.
- Lets say i want to display card inside user info screen and products screen
 - But content & format of the content inside card will be different for these 2 screens
 - lets say in users screen card should contain username, age and mobile number. Whereas in card in products page should display product name, price and sale price.
 - How its possible ? by using children props its possible.
 - I will create a common card layout and i will send the content dynamically from user component and products component.

In real time applications, Layout will be same for all the screens. like Header, footer, sidenav will be same. We will be adding these Header, sidenav and footer in layout component and the middle part is dynamic which will be displayed dynamically using children props.

```
import { Card } from "./Card";

export const UserInfo = () => {
  return (
    <Card>
      >
        Screen: UserInfo
        <h3>Name: Saikrishna</h3>
        <h3>Mobile: 7778877788</h3>
      </Card>
    );
};
```

===== Products component =====

```
import { Card } from "./Card";
```

```
export const Products = () => {
  return (
    <Card>
      <h3>Product Name: Cycle</h3>
      <h3>Date of manufacture: Jan 10th</h3>
    </Card>
  );
};
```

===== Card component =====

```
export const Card = ({ children }) => {
  return (
    <div
      style={{
        border: "1px solid gray",
        borderRadius: "8px",
        padding: "16px",
        marginBottom: "10px",
      }}
    >
      {children}
    </div>
  );
};
```

State vs Props:

- State is used to store the components data and when ever state updates, UI will gets rerendered. whereas, props are used to send the data from one component to another component.
 - State Can be changed whereas props cannot be changed
-

Interview Questions & Answers:

1. What are props ?

Answer:

- Props are used to send the data from one parent component to child component.
 - Props are immutable. We will not be able to update the props in child component.
-

2. Can we pass data from child to parent using props?

Answer:

- No, props support **one-way data flow** (parent → child).
 - But we can achieve child-to-parent communication by **passing a function as a prop** from parent to child and calling it inside the child
-

3. What is default props?

Answer:

Default props are **fallback values** used when the parent component does not send a prop.

They prevent blank UI or errors.

4. What is the `children` prop?

Answer:

`children` is a special prop that allows us to pass **JSX content** inside a component.

It is mainly used to create reusable layout components.

5. What is the difference between props and state?

Answer:

- State is used to store the components data and when ever state updates, UI will gets rerendered. whereas, props are used to send the data from one

component to another component.

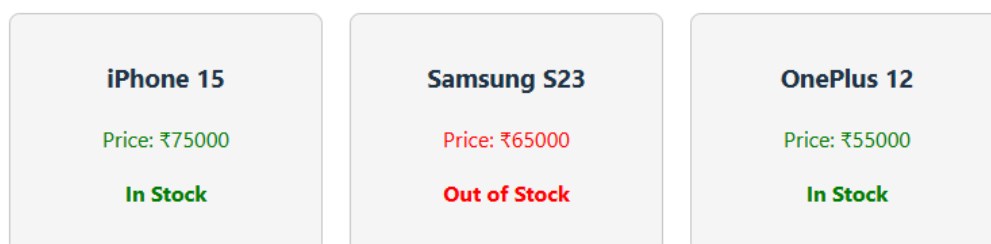
- State Can be changed whereas props cannot be changed
-

Mini Assignments:

1. Build a Product List using Props

- Create **two components**: `Products` and `ProductCard`
- Create a **products array** inside the `Products` component
- Pass the **products data to the child component (`ProductCard`) using props**
- In `ProductCard`, use **`map()`** to display each product
- Each product card should show **product name, price, and stock status**
- Use **conditional rendering** to show **"In Stock"** or **"Out of Stock"**
- Apply **card styling using CSS Modules** to make the UI clean and neat

Expected UI :



2. Implement a Like & dislike button for a post or product:

- Create a **Parent component** that will hold the state for `likeCount` and `dislikeCount`.
- Create a **Child component** called `LikeDislikeButtons` that will display:
 - A **Like button**
 - A **Dislike button**
 - Both buttons should show the current count next to them.
- The **functions to handle like and dislike clicks** should be passed from the parent to the child via **props**.
- Clicking **Like** should **increase the like count**, clicking **Dislike** should **increase the dislike count**.
- Apply some **basic styling** to make the buttons visually appealing.

Expected UI:

React Like & Dislike Example



Hooks:

- Hooks are special functions provided by reactjs which allows us to use react features like.,
 - Using State inside a component
 - We can access lifecycle events

- We can call Api calls
- We can perform side effects
- We can optimize performance of our application
- We can access dom elements.
- Hooks were introduced in 16.8 version of Reactjs. Before 16.8 version, All these features were available only inside class components so developers used to use class components to create components in reactjs. But after introduction of hooks, all the developers started migrating towards function components.
- Now even react official documentation itself is recommending developers to use function components instead of class components.

Different Hooks in Reactjs:

- Basic hooks:
 - useState
 - useEffect
 - useContext
- Advanced:
 - useRef
 - useReducer
 - usememo
 - usecallback
- Rare(Advanced Hooks):
 - useLayoutEffect
 - useImperativeHandle
 - useDebugValue

Interview Questions & Answers:

1. What are Hooks in React?

Answer:

Hooks are special functions provided by React that allow us to use React features like state, lifecycle events, API calls, side effects, performance optimization, and DOM access inside function components.

2. Tell some commonly used hooks in function components ?

Answer:

- useState
 - useEffect
 - useRef
 - useContext
 - useReducer
 - useMemo
 - useCallback
-

UseEffect:

- In reactjs, useEffect is used to perform some sideeffects inside a function component.
- These sideeffects will be executed once component is rendered.
- Examples: Api calls, setting timers, logging, doing some analytics, subscribing to events etc.

SideEffects:

- Any thing which is not part of Ui rendering(not returns jsx) is called a sideeffect.

Examples:

- Api calls:
 - We use fetch to call external apis. This will not return any jsx This is an external operation. . so this is a side effect.

- Even though result is displayed in the ui later, the action is outside ui rendering.
 - 👉 UI is shown **without API data initially we will show loaders.**
 - Changing browser.title:
 - Here we are changing title of page. which is not part of ui so this is a sideeffect.
 - Setting timers using setTimeout:
 - This sets timer in browser which is not part of ui.
 - Logging /Analytics
 - `console.log("User logged in");`
 - This is not part of ui.
 - Subscribing to some events
 - `window.addEventListener("resize", () => {});`
-

Let us take some simple example.,

- Lets us take an ecommerce application, We want to display products list. We will fetch the product list from api. We will call this api call in useEffect.
 - Lets take another example, We are in quiz application. When we open some quiz it will show some timer to us. So when component is rendered they are starting some timer. So where they are doing this ? They are starting timer in useeffect.
-

Component Life cycle :

There are 3 different stages in life cycle of a component. We can handle all these life cycle events by using useeffect.

1. Mounting phase
 - a. During this phase component will be mounted into the DOM.
2. Updating phase
 - a. During this phase component will be updated in the DOM

3. Unmounting phase.

- a. During this phase component will be removed from the dom.

Mounting phase:

We will handle this using useeffect with empty dependency array.

useEffect with empty dependency array :

- When ever we want to execute some logic once the component is rendered for the first time, then we will be using useeffect with empty dependency array.
 - Doing api calls.
 - Setting timers
 - Subscribing to some events etc.

Syntax:

```
useEffect(()=>{  
  
  // logic  
  
},[])
```

Lets take some quiz application example where timer starts on initial render.

```
import { useEffect, useState } from "react";  
  
function Parent() {  
  const [seconds, setSeconds] = useState(0);  
  
  useEffect(() => {  
    const timerId = setInterval(() => {  
      setSeconds(prev => prev + 1);  
    }, 1000);  
  }, []);  
}
```

```

    // cleanup
    return () => {
      clearInterval(timerId);
    };
  }, []);

  return (
    <>
      <h1>Timer: {seconds} seconds</h1>
    </>
  );
}

export default Parent;

```

Updating Phase:

We can handle this phase using `useEffect` with dependency array.

1. useEffect with some dependency array:

Syntax:

```
useEffect(() => {}, [count]);
```

This is very very important and very commonly used in real time applications.

- Here when ever the values inside the dependency array changes, the logic inside `useEffect` will be executed.

Example:

- In real time ecommerce applications, Some times we need to execute the logic inside our component based on.,
 - some prop change or
 - based on some value of global redux variable.
- We can achieve this by using `useEffect` with dependency array (In dependency array we will add that prop or global variable as

dependency)

- **Explanation :**

- In this example we will have two buttons "electronics" and "clothing". Now based on these selections we will display message to the user.
- Show by debugging when button clicked debug point moves to useeffect.
- If same button is clicked, debug will not be called.

```
import { useEffect, useState } from "react";

function Parent() {
  const [message, setMessage] = useState("");
  const [sel, setSel] = useState("");

  useEffect(() => {
    setMessage(`Hey you have selected ${sel}`);
  }, [sel]);

  return (
    <>
      <button
        onClick={() => {
          setSel("Electronics");
        }}
      >
        Electronics
      </button>
      <button
        onClick={() => {
          setSel("Clothing");
        }}
      >
        Clothing
      </button>

      <h1>{message}</h1>
    </>
  )
}
```

```

    );
  }

  export default Parent;

```

2. useEffect with multiple dependencies:

```
useEffect(() => {}, [dependency1, dependency2]);
```

- Dependency array can take multiple values aswell. When ever any of those dependencies changes, the logic inside useeffect will be executed.
- Lets say i want to display user dynamically and category dynamically.

```

import { useEffect, useState } from "react";
import "./App.css"

function Parent() {
  const [category, setCategory] = useState("");
  const [userType, setUserType] = useState("");
  const [message, setMessage] = useState("");

  useEffect(() => {

    if (category && userType) {
      setMessage(
        `Hey ${userType}, you selected ${category}`
      );
    }
  }, [category, userType]); // 👉 multiple dependencies

  return (
    <>
      <h2>Select User Type</h2>

      <button onClick={() => setUserType("Sai")}>
        Sai
      </button>
    </>
  );
}

```

```

    <button onClick={() => setUserType("Krishna")}>
      Krishna
    </button>
    <h2>Select Category</h2>

    <button onClick={() => setCategory("Electronics")}>
      Electronics
    </button>

    <button onClick={() => setCategory("Clothing")}>
      Clothing
    </button>

    <h1>{message}</h1>
  </>
);
}

export default Parent;

```

Unmounting Phase:

We can handle this phase by using clean up function in useeffect.

Clean up function in useEffect:

```

useEffect(() => {

  return () => {
    // Cleanup logic
  };
}, []);

```


- Sometimes we want to execute some code when navigates to another component or when component is destroyed, we use these cleanup functions.
- Mostly its used for some cleanup activities like,
 - Clearing timers like setTimeout & setInterval
 - Clearing event listeners
 - Clearing all subscriptions etc.
 - Cleaning websockets.

Example Explanation:

- Lets create some sample example, Lets create some component called flashsale and a button to stop flashsale.
- When user clicks on button, flash sale component will be removed from dom. and when its removed from dom we will perform some clean up activities.
- Now when i click on button, we can see a log called "Timer stopped".
- Deubg will go inside cleanup return statement before useEffect reruns and when component is destroyed.

```
import { useEffect, useState } from "react";

function FlashSale() {
  useEffect(() => {
    return () => {
      console.log("Timer stopped");
    };
  }, []);

  return <h2>Flash Sale Started</h2>;
}

function Parent() {
  const [show, setShow] = useState(true);

  return (
```

```

    <div>
      <button onClick={() => setShow(false)}>Leave Sale Page</button>
      {show && <FlashSale />}
    </div>
  );
}

export default Parent;

```

Indetail example:

```

import { useEffect, useState } from "react";

function FlashSale() {
  const [seconds, setSeconds] = useState(10);

  useEffect(() => {
    const timer = setInterval(() => {
      setSeconds(prev => prev - 1);
    }, 1000);

    return () => {
      clearInterval(timer);
      console.log("Timer stopped");
    };
  }, []);

  return <h2>Flash Sale Ends In: {seconds}s</h2>;
}

function Parent() {
  const [show, setShow] = useState(true);

  return (

```

```

    <div>
      <button onClick={() => setShow(false)}>Leave Sale Page</button>
      {show && <FlashSale />}
    </div>
  );
}

export default Parent;

```

No dependency array :

- Runs on initial render of the component
- After every state update

This is very very rarely used. As there is no dependency array, it will run on every minute state update so it causes performance issues.

How to Explanation:

First remove onclick method in button and show useeffect does not runs. Once onclick is placed and state is updated, it will run useeffect.

```

import { useEffect, useState } from "react";

function Parent() {
  const [count, setCount] = useState(0);

  useEffect(() => {
    console.log("render");
  });

  return (
    <>
      <h1>{count}</h1>
      <button
        onClick={() => {

```

```
        setCount((count) => count + 1);
      }}
    >
      Click
    </button>
  </>
);
}

export default Parent;
```

Mini Assignments:

1. Fetching products & Filter by category

◆ Requirement

- Fetch products from following url
 - <https://fakestoreapi.com/products>
- Display all products
- Create category buttons dynamically from API data
- On category button click:
 - Show only that category products
 - Highlight active category
- If same category is clicked again → do nothing

Expected Result:

Category Filter

All

men's clothing

jewelery

electronics

women's clothing

Fjallraven - Foldsack No. 1 Backpack, Fits 15 Laptops

Category: men's clothing

₹ 109.95

Mens Casual Premium Slim Fit T-Shirts

Category: men's clothing

₹ 22.3

Mens Cotton Jacket

Category: men's clothing

₹ 55.99

Mens Casual Slim Fit

Category: men's clothing

₹ 15.99

Solution:

Try on your own.

2. Implement Price Range & Category filter on Products

◆ Requirement

- Dropdown to select category (All categories from products data)
- Two inputs: **Min price** and **Max price**
- Filter products based on:
 - Selected category

- Price range
- Show "No products found" if result empty

Expected Result:

Product Filter

jewellery ▼

Min Price

Max Price

John Hardy Women's Legends Naga Gold & Silver Dragon Station Chain Bracelet

Category: jewellery

₹ 695

Solid Gold Petite Micropave

Category: jewellery

₹ 168

White Gold Plated Princess

Category: jewellery

₹ 9.99

Pierced Owl Rose Gold Plated Stainless Steel Double

Category: jewellery

₹ 10.99

Solution:

Try on your own.

3. Implement Sorting filters (Additional to above filters)

◆ Requirement

Add sorting options:

- Price: Low → High
- Price: High → Low
- Rating: High → Low
- Popularity (rating.count)

Sort should work **with filters applied**

Expected Result:

Product Filter & Sort

jewellery ▼

Min Price

Max Price

Popularity ▼

John Hardy Women's Legends Naga Gold & Silver Dragon Station Chain Bracelet
Category: jewellery
Price: ₹ 695
Rating: ★ 4.6
Popularity: 400

White Gold Plated Princess
Category: jewellery
Price: ₹ 9.99
Rating: ★ 3
Popularity: 400

Pierced Owl Rose Gold Plated Stainless Steel Double
Category: jewellery
Price: ₹ 10.99
Rating: ★ 1.9
Popularity: 100

Solution:

Try on your own.

Interview Questions & Answers

1. What is `useEffect` in React?

Answer:

`useEffect` is a React Hook used to perform **side effects** in functional components.

Side effects are operations that are **outside UI rendering**, such as:

- API calls
 - Timers
 - Event subscriptions
 - Logging / analytics
 - DOM updates like changing document title
-

2. What are side effects in React?

Answer:

Any operation that **does not return JSX** and **interacts with the outside world** is a side effect.

Examples:

- Fetching data from an API
 - Setting timers (`setTimeout` , `setInterval`)
 - Subscribing to events
 - Logging data
 - Updating browser title
-

3. Why can't we do side effects directly inside a component body?

Answer:

React may re-render components multiple times.

If side effects are written directly inside the component:

- API calls may repeat
- Timers may be duplicated
- Event listeners may leak

`useEffect` ensures side effects run **at the correct lifecycle stage**.

4. When does `useEffect` run?

Answer:

`useEffect` runs **after the component is rendered and painted on the UI**.

Execution order:

1. Component function runs
2. Virtual DOM is created
3. Real DOM updates
4. UI is shown
5. `useEffect` executes

◆ DEPENDENCY ARRAY QUESTIONS

5. What is the dependency array in `useEffect` ?

Answer:

The dependency array controls **when the effect runs**.

React re-runs the effect **only when values in the dependency array change**.

6. What happens if dependency array is empty `[]` ?

Answer:

The effect runs **only once** after the first render (Mounting phase).

Common use cases:

- API calls
- Timers
- Event listeners
- Initial setup logic

```
useEffect(() => {  
  fetchData();  
}, []);
```

7. What happens if dependency array has values?

Answer:

The effect runs:

- Once on initial render
- Again whenever **any dependency changes**

```
useEffect(() => {  
  console.log(category);  
}, [category]);
```

8. Can `useEffect` have multiple dependencies?

Answer:

Yes.

If **any dependency changes**, the effect runs again.

```
useEffect(() => {  
  updateMessage();  
}, [category, userType]);
```

9. What happens if we don't pass a dependency array?

Answer:

The effect runs:

- After initial render
- After **every re-render**

This is **rarely used** because it causes **performance issues**.

◆ LIFECYCLE RELATED QUESTIONS

10. How do we handle component lifecycle in functional components?

Answer:

Using `useEffect`.

Lifecycle Phase	useEffect Usage
Mounting	<code>useEffect(() => {}, [])</code>
Updating	<code>useEffect(() => {}, [deps])</code>
Unmounting	Cleanup function

11. How do you handle unmounting in `useEffect` ?

Answer:

By returning a **cleanup function** from `useEffect`.

```
useEffect(() => {  
  const timer = setInterval(() => {}, 1000);  
  
  return () => {  
    clearInterval(timer);  
  };  
}, []);
```

12. When does cleanup function run?

Answer:

Cleanup function runs:

1. Before the component is unmounted
 2. Before the effect re-runs due to dependency change
-

13. Why is cleanup important?

Answer:

It prevents:

- Memory leaks
 - Duplicate timers
 - Multiple event listeners
 - Open WebSocket connections
-

◆ PRACTICAL / SCENARIO QUESTIONS

14. Where do you make API calls in React and why?

Answer:

Inside `useEffect` with an empty dependency array.

Reason:

- Prevent multiple API calls
 - Run only after component is rendered
-

15. Why do we show loaders while API data is fetching?

Answer:

Because:

- UI renders **before API response**
 - Data arrives asynchronously
 - Loader gives better user experience
-

16. What happens if we update state inside `useEffect` ?

Answer:

- Component re-renders
- If dependency is missing → infinite loop
- Proper dependency array prevents issues

17. What causes infinite loop in `useEffect` ?

Answer:

Updating a state inside `useEffect` **without proper dependency array**.

✗ Example:

```
useEffect(() => {  
    setCount(count + 1);  
});
```

18. Can `useEffect` replace class lifecycle methods?

Answer:

Yes.

Class Lifecycle	useEffect Equivalent
componentDidMount	<code>useEffect(() => {}, [])</code>
componentDidUpdate	<code>useEffect(() => {}, [deps])</code>
componentWillUnmount	Cleanup function

◆ ADVANCED / TRICK QUESTIONS

19. Does `useEffect` run before or after render?

Answer:

After render and after DOM updates.

20. Can we have multiple `useEffect` s in one component?

Answer:

Yes.

Recommended for **separation of concerns**.

```
useEffect(() =>fetchData(), []);  
useEffect(() =>updateTitle(), [count]);
```

21. Why React Strict Mode runs `useEffect` twice in development?

Answer:

React Strict Mode **intentionally double-invokes effects** in development to:

- Detect side effects
- Find memory leaks
- Ensure cleanup logic works correctly

(Not in production)

22. Should async function be passed directly to `useEffect` ?

Answer:

No.

 Wrong:

```
useEffect(async () => {});
```

 Correct:

```
useEffect(() => {  
  asyncfunctionfetchData() {}  
  fetchData();  
}, [])
```

UseRef:

Why useRef is needed ?

1. Previously before reactjs, means in plain javascript if we want to access the dom element we have various DOM selection methods like.,

- document.getElementById
- document.getElementsByClassName
- document.querySelector and many more.

Using these methods, developers manually finds the elements and updates them.

- Whereas, In Reactjs, Mostly all the DOM updates are automatically handled by Reactjs itself..
- But if we want to access the DOM elements, we can use a hook called useRef. This will not rerender the component.

2. If we want to store the value between rerenders without causing component rerender

Examples 1:

- If you open product details page in an ecommerce application, we will have star ratings, When user clicks on rating, it will scroll down to reviews section. (**Show them amazon product details rating example**)
- So we can achieve that by using useRef.

```
import { useEffect, useRef } from "react";

function Parent() {
  const reviewRef = useRef();

  return (
    <>
      <button onClick={() => reviewRef.current.scrollToView()}>
        Go to Reviews
      </button>

      <div style={{ backgroundColor: "yellow", height: "1000px" }}>Product Details</div>
    </>
  );
}
```

```

        <div style={{ backgroundColor: "red", height: "1000p
x" }} ref={reviewRef}>
          ★★★★★ Reviews
        </div>
      </>
    );
  }

  export default Parent;

```

Example 2:

In realtime swiggy search, searchbox is auto focussed on page load. How its implemented ? By using useRef.

<https://www.swiggy.com/search>

```

import { useEffect, useRef } from "react";
import "./App.css";

function App() {
  const inputRef = useRef();

  useEffect(() => {
    inputRef.current.focus();
  }, [])

  return (
    <>
      <input ref={inputRef}/>
    </>
  );
}

export default App;

```


In Interviews: How to answer where you have used in your project:

- They will ask candidate about where they have used useRef in their application.
- Tell that in my application i have search page where user can search for all the products. and when user comes to search page we need to autofocus the search element.
- In my application i have reviews section at the bottom and reviews navigation button in top when user clicks on reviews navigation button we navigated to reviews section using this useRef.
- Lets say i have textbox element and i need to show number of characters left and when user enters text it should update characters left count . so for this if i use useState, it will rerender component on every character change. but if i use useRef it will not rerender the component everytime.

Component will not be rerendered when we use useRef:

I am increment count using useState and useRef. When i click on useState button its rerendering component but when i click on useRef button its not rerendering the component.

```
import { useState, useRef } from "react";
import "./App.css";

function App() {
  const [count, setCount] = useState(0);
  const countRef = useRef(null);

  console.log("component rerendered");
  return (
    <>
      <button onClick={() => setCount(count + 1)}>useState
    </button>
    <button
      onClick={() =>
        (countRef.current.innerHTML = Number(countRef.cur
```

```

    rent.innerHTML) + 1)
    }
    >
    useRef{" "}
  </button>

  <h2>UseRef </h2>
  <h2 ref={countRef}>0</h2>
  <h1></h1>
  <h2>Uestate </h2>
  <h2>{count}</h2>
</>
);
}

export default App;

```

Forward Ref: (This is not needed in React 19 version)

- Lets say i have an input element in child component and a button in parent component.
- Now when user clicks on parent component button, input inside child component should be auto focussed. How we can achieve this ?
 - We can use Forward Ref's for this.

Forward refs are used to access the child component ref's in parent component.

```

import { useEffect, useRef } from "react";
import Child from "./Child";

function Parent() {
  const inputRef = useRef();

  return (
    <>

```

```

    <button
      onClick={() => {
        inputRef.current.focus();
      }}
    >
      Focus on Input
    </button>

    <Child ref={inputRef}></Child>
  </>
);
}

```

```
export default Parent;
```

```

===== Child component =====
=====

```

```

import { forwardRef } from "react";

const Child = forwardRef((props, ref) => {
  return (
    <>
      <input ref={ref} {...props} />
    </>
  );
});

export default Child;

```

- Here props and refs are mandatory. If we don't want props use empty object like below.,
 - `forwardRef(({ }, ref) => {`

```
import { forwardRef } from "react";
```

```
const Child = forwardRef(( { } , ref) => {
  return (
    <>
      <input ref={ref} {...props} />
    </>
  );
});

export default Child;
```

useImperativeHandle:

- It is used for encapsulation. This will expose some api to parent component. By using this child will have control over child DOM. parent will not be able to directly access the dom elements.
- What can be accessed by parent will be decided by child component using useImperativeHandle hook.

Where in real time ?

If our comment element is used in multiple applications and you want to restrict what can be accessed by parent component.

`useImperativeHandle` takes **3 arguments**:

```
useImperativeHandle(ref, createHandle, [deps])
```

- `ref` → The ref passed from the parent
- `createHandle` → Function that returns an object containing methods/properties to expose
- `[deps]` → Optional dependency array (like `useEffect`)

Let's add methods to the child:

```
const Input = (props, ref) => {
  const inputRef = useRef();
```

```

    useImperativeHandle(ref, () => ({
      focus() {
        inputRef.current.focus();
      },
      clear() {
        inputRef.current.value = "";
      },
      setValue(value) {
        inputRef.current.value = value;
      },
    }));

    return <input ref={inputRef} />;
  };

export default Input;

```

Now the parent cannot access inputRef directly, but can call focus(), clear(), and setValue().

```

import { useRef } from "react";
import Input from "../Input";

function App() {
  const inputApiRef = useRef(); // parent ref

  return (
    <>
      <Input ref={inputApiRef} />

      <button onClick={() => inputApiRef.current.focus()}>Focus Input</button>
      <button onClick={() => inputApiRef.current.clear()}>Clear Input</button>
      <button onClick={() => inputApiRef.current.setValue("Hello")}>Set Value</button>
    </>
  );
}

```

```
);  
}  
  
export default App;
```

Interview questions & answers:

1. What is useRef ?

Ans:

- If we want to access the dom element in reactjs we use useRef.
- It will be used to store the data with out causing component rerender and persists the data across component rerenders.

2: What are the differences between `useRef` and `useState`

Answer:

Feature	<code>useRef</code>	<code>useState</code>
Triggers render	✗ No	✓ Yes
Persists value	✓ Across renders	✓ Across renders
Typical use case	DOM access, mutable variables	UI state

3: What is `forwardRef` ?

Answer:

`forwardRef` is a React API used to forward a ref from a parent component to a child component, allowing the parent to directly access the child's DOM or methods.

In React 19, We can send ref as a prop. so forward ref is not required from React19 version and it will be deprecated soon.

4. What is `useImperativeHandle` ?

Answer:

`useImperativeHandle` allows you to expose **specific functions or values** instead of the whole DOM node. Child component decides what things parent component can access.

5. How does `useImperativeHandle` improve encapsulation?

- It allows you to hide DOM elements and expose only the methods you want.
-

6. Can you use `useRef` to trigger re-render?

- No. `useRef` does not trigger re-renders. For UI updates, use `useState`.
-

Real time Mini Assignments:

1. Implement scroll to top functionality using `useRef`

2. Implement File upload trigger (Hidden input)

📌 **Use case:** Custom Upload Button

React Router:

In this session we are going to cover,,

- Browser router
- Routes & Route
- Link tag
- Programmatic navigation

- Dynamic routing using Url parameters
 - Wild card routing
 - Layout routes & Nested Routes
 - Nested Pages
 - Protected Routes
 - Navigationn State
-

- React router will enable client side routing in our react application.
- In a **traditional multi-page website**, whenever we navigate from one page to another, the browser **reloads the entire page** and sends a request to the server to fetch a **new HTML document**.
- Whereas in **Single Page Applications (SPAs)** built using **ReactJS** load **only one HTML file** (`index.html`) initially. After that, the application **does not reload the page** and **does not request a new HTML file from the server** during navigation.
- Even though the page does not reload, we are still able to see **different screens or components** when we navigate.

👉 This happens because of **React Router**. It will handle the routing in our react application.

React Router listens to changes in the **browser URL** and, based on the configured **paths**, it **decides which React component should be rendered**.

This component switching happens **inside the browser itself**, without a full page reload, giving us a fast and smooth user experience.

Installation:

```
npm install react-router-dom
```

Core Building blocks of React router:

Browser Router:

- Without Browser router, routing will not work.
- It uses browser history api and enables routing in our react application.
- We will wrap entire app component in Browser router.

```
import { createRoot } from "react-dom/client";
import App from "./App.jsx";
import { BrowserRouter } from "react-router";

createRoot(document.getElementById("root")).render(
  <BrowserRouter>
    <App />
  </BrowserRouter>
);
```

Routes & Route :

- This will contain mapping between url paths and components.
- Based on this mapping browser decide which component to be rendered for a given url.
- Here Routes will act as a container for Route mappings.

```
import { Link, Route, Routes } from "react-router";
import { Home } from "./Components/Home";
import Products from "./Components/Products";

function App() {
  return (
    <>
      <nav>
        <Link to="/">Home</Link>
```

```

        <Link to="/Products">Products</Link>
    </nav>

    <Routes>
        <Route path="/" element={<Home />}></Route>
        <Route path="/Products" element={<Products />}></Ro
ute>
    </Routes>
</>
);
}

export default App;

```

- Here in path, we need to specify the url path and in element we need to provide component name.

Link tag: (JSX navigation)

- This is used for navigation. It will changes the url **with out reload**. whereas anchor tag will changes the url with full page reload
- Link tag can be used only inside jsx.

```

import { Link, Route, Routes } from "react-router";
import { Home } from "../Components/Home";
import Products from "../Components/Products";

function App() {
    return (
        <>
            <a href="/Products">Products</a>
            <nav>
                <Link to="/">Home</Link>
                <Link to="/Products">Products</Link>
            </nav>
        </>
    );
}

```

```

    <Routes>
      <Route path="/" element={<Home />}></Route>
      <Route path="/Products" element={<Products />}></Ro
ute>
    </Routes>
  </>
);
}

export default App;

```

Programatic navigation:

- Some times we want to navigate based on button click action.
- In an ecommerce application, lets say iam displaying all products in home page. Now when user clicks on any product, i should be navigated to product details page. How we can achieve this ?
- Navigate after form submission
- Navigate after API success or failure.
- Redirect after login/logout
- Navigate to product details page

Here we will use useNavigate function from react router for programattic navigation.

App.jsx :

```

<Routes>
  <Route path="/" element={<Home />}></Route>
  <Route path="/Products" element={<Products />}></Route>
  <Route path="/ProductDetails" element={<ProductDetails/>}
></Route>

```

```
</Routes>
```

Products.jsx :

```
import { useNavigate } from "react-router";

const Products = () => {
  const navigate = useNavigate();

  const handleNavigation = async () => {
    navigate("/ProductDetails");
  };

  return (
    <div>
      <button onClick={handleNavigation}>Product details page</button>
    </div>
  );
};

export default Products;
```

ProductDetails.jsx :

```
export const ProductDetails = () => {
  return <h1>Product Details Page</h1>;
};
```

Navigation State:

- If we want to send some temporary data from one route to another route during navigation. Then we will use this navigation state.
- This information will not be displayed in the url.

- If we want to send data from products page to products details page

Products component:

```
import { useNavigate } from "react-router";

export const Products = () => {
  const navigate = useNavigate();

  const products = [
    { id: 1, name: "bottle" },
    { id: 2, name: "Bag" },
  ];

  const handleNavigation = (prod) => {
    const prods = {
      prodId: prod.id,
      prodName: prod.name,
    };
    navigate("/ProductDetails", { state: prods });
  };

  return (
    <>
      <h1>Products</h1>
      {products.map((prod) => {
        return (
          <>
            <h1>{prod.name}</h1>
            <button
              onClick={() => {
                handleNavigation(prod);
              }}
            >
              Navigate{" "}
            </button>
          </>
        );
      })}
    </>
  );
}
```

```

    }
  }
  </>
);
};

```

Product details component:

```

import { useLocation } from "react-router";

export const ProductDetails = () => {
  const location = useLocation();

  return <h1>Product Details {location.state.prodName}</h1>
  >;
};

```

Dynamic Routing: (URL Parameters)

- Some times we want to send parameters in url then we can use this dynamic routing.
- Lets say i have products page and product details page, When user clicks on any product, i will navigate to productdetail page and during this navigation i will send productid so that in product details page i will fetch the product details based on productid.

Here we will use useParams function from react router to get the parameter from url.

App.js

```

<Route path="/ProductDetails/:id" element={<ProductDetails/>}></Route>

```

Products Component :

```

import { useNavigate } from "react-router";

const Products = () => {
  const navigate = useNavigate();

  const productsInfo = [
    { id: 1, name: "water bottle" },
    {
      id: 2,
      name: "bag",
    },
  ];

  const handleProductNavigation = (id) => {
    navigate(`/ProductDetails/${id}`);
  };

  return (
    <div>
      {productsInfo.map((prod) => {
        return (
          <>
            <h1>{prod.name}</h1>
            <button onClick={() => {handleProductNavigation(p
rod.id)}}>
              Product Detail
            </button>
          </>
        );
      })}
    </div>
  );
};

```

Product Details Component:

```
import { useParams } from "react-router";

export const ProductDetails = () => {
  const {id} = useParams();
  return <h1>Product Details Page {id}</h1>;
};
```

Lets say i want to send multiple parameters. Can i send it using this dynamic params ?

- Yes you can but with slight modifications

App.jsx

```
<Route path="/ProductDetails/productId/:productId/productName/:productName"
  element={<ProductDetails/>}></Route>
```

Products component:

```
import { useNavigate } from "react-router";

const Products = () => {
  const navigate = useNavigate();

  const productsInfo = [
    { id: 1, name: "water bottle" },
    {
      id: 2,
      name: "bag",
    },
  ];

  const handleProductNavigation = (id, name) => {
```



```

        navigate(`/ProductDetails/productId/${id}/productName/
${name}`);
    };

    return (
        <div>
            {productsInfo.map((prod) => {
                return (
                    <>
                        <h1>{prod.name}</h1>
                        <button onClick={()=>{handleProductNavigation(p
rod.id,prod.name)}}>Product Detail</button>
                    </>
                );
            })}
        </div>
    );
};

export default Products;

Products detail :

import { useParams } from "react-router";

export const ProductDetails = () => {
    const {productId, productName} = useParams();
    return <h1> {productId}{productName}</h1>;
};

```

Wild card Routing (404):

- Some user tries to manually enter url and if it does not matches our routes map, then we will navigate user to 404 page saying that page does not exist.

App.jsx

```
import "./App.css";
import { Link, Route, Routes } from "react-router";
import { Home } from "../Components/Home";
import { NotFound } from "../Components/NotFound";

function App() {
  return (
    <>
      <Routes>
        <Route path="/" element={<Home />}></Route>
        <Route path="*" element={<NotFound/>}></Route>
      </Routes>
    </>
  );
}

export default App;
```

NotFound component:

```
export const NotFound = () => {
  return (
    <div>
      <h1>404</h1>
      <p>Page Not Found</p>
    </div>
  );
};
```

Layout Routes & Nested Routes:

If you look at a website like Myntra, there is a header with navigation links like Men, Women, Kids, Home, etc., and a footer with additional information.

When a user navigates to pages like Men, Women, or Home, the respective page content is displayed. But the header and footer remain consistent across all pages.

How it is implemented (React + React Router way)

We do not copy the header and footer code in every page component. That would be repetitive and hard to maintain.

Instead, a common layout component is created, e.g., MainLayout. This layout contains:

- Header
- Footer
- Optional Sidebar / Side Navigation

The main content area in the middle is dynamic. For this, React Router v6 provides nested routes and the `<Outlet>` component.

Folder Structure

```
src/  
├─ layouts/  
│   └─ DashboardLayout.jsx  
├─ pages/  
│   ├─ DashboardHome.jsx  
│   ├─ Profile.jsx  
│   ├─ Orders.jsx  
│   └─ NotFound.jsx  
└─ App.jsx
```

Dashboard Layout (Layout Route)

```

import { Link, Outlet } from "react-router-dom";

const Dashboard = () => {
  return (
    <>
      <header
        style={{
          backgroundColor: "yellow",
          width: 1000,
          padding: 50,
        }}
      >
        <Link style={{padding: 20}} to="/Products">Products
</Link>
        <Link style={{padding: 20}} to="/Orders">Orders</Link>
      </header>

      <div style={{
        height: 500,
        backgroundColor: "green"
      }}>
        <Outlet></Outlet>
      </div>

      <footer style={{backgroundColor:"red",
        padding: 10
      }}><h1>Footer </h1></footer>
    </>
  );
};

export default Dashboard;

```

Nested Pages

```
// pages/DashboardHome.jsx
export default function DashboardHome() {
  return <h3>Welcome to Dashboard Home</h3>;
}

const Orders = () => {
  return <h1>Orders</h1>
}
export default Orders;

const Products = () => {
  return <h1>Products</h1>
}
export default Products;

// pages/NotFound.jsx
export default function NotFound() {
  return <h3>404 - Page Not Found</h3>;
}
```

App.jsx – Routing Setup

```
import { Route, Routes } from "react-router-dom";
import "./App.css";
import Dashboard from "./Components/Dashboard";
import Products from "./Components/Products";
import Orders from "./Components/Orders";
import Home from "./Components/Home";

export default function App() {
  return (
    <>
      <Routes>
        <Route path="/" element={<Dashboard></Dashboard>}>

```

```

        <Route index element={<Home></Home>}></Route>
        <Route path="Products" element={<Products></Products>}></Route>
        <Route path="Orders" element={<Orders></Orders>}></Route>
    </Routes>
</>
);
}

```

Protected Routes:

- In most of the enterprise applications most of the pages like admin/profile/Account/Orders will be restricted. Only authenticated users will be able to access those pages.
- If user is loggedin then we will display those pages else we will redirect user to login page
- How to achieve this ?
 - By using protected routes.

Create a fake flipkart account and try to navigate to orders page by using below url

https://www.flipkart.com/account/orders?link=home_orders

if loggedin it will navigate to orders page. else to login page

Explain by adding Dashboard directly in validateUser component. Then create another Admin component and make the validateUser component reusable.

```
// App component

import { Route, Routes } from "react-router";
import "./App.css";
import { ValidateUser } from "../Components/ValidateUser";
import { Dashboard } from "../Components/Dashboard";
import { Admin } from "../Components/Admin";

function App() {
  return (
    <Routes>
      <Route path="/dashboard" element={<ValidateUser>
        <Dashboard/>
      </ValidateUser>}></Route>
      <Route path="/Admin" element={<ValidateUser>
        <Admin/>
      </ValidateUser>}></Route>
    </Routes>
  );
}

export default App;
```

Validate User component:

```
import { Login } from "../Login";

export const ValidateUser = ({children}) => {
  const isLoggedIn = false;

  return <>{isLoggedIn ? children : <Login />}</>;
};
```

```
// Admin component
export const Admin = () => {
  return <h1>Admin</h1>
}

// Dashboard component
export const Dashboard = () => {
  return <h1>Dashboard</h1>
}

// Login component
export const Login = () => {
  return <h1>Login</h1>
}
```

Mini Assignment:

a. Build Basic E-commerce Routing

- Create routes:
 - `/`
 - `/products`
 - `/product/:id`
 - `/cart`
 - `/orders`
- Add Navbar with `NavLink`
- Highlight active tab

b. Product Details Page

- On clicking product → navigate to `/product/:id`
- Display Product data in console

- Handle invalid product ID

c. Implement Bread crumb navigation in ecommerce site

d. Implement a Protected Checkout Route

- Protect `/checkout` route
- If not logged in → redirect to `/login`
- After login → redirect back to checkout

e. Nested Routes – Account Page

Task

- `/account`
 - `/account/profile`
 - `/account/orders`
 - `/account/address`
- Use `Outlet`
- Highlight active sub-route

Interview Questions & Answers

1 What is React Router?

Answer:

React Router is a library that enables **client-side routing** in React applications. It allows navigation between different components **without reloading the page**.

2 What problem does React Router solve?

Answer:

It solves the problem of:

- Full page reloads
 - Fetching new HTML files on every navigation
- by enabling **Single Page Application (SPA)** navigation.
-

3 What is the difference between a traditional website and an SPA?

Answer:

Traditional Website	SPA
Multiple HTML files	Single HTML file
Page reload on navigation	No page reload
Server handles routing	Client handles routing
Slower navigation	Faster navigation

4 How does React Router work internally?

Answer:

React Router:

- Listens to **URL changes**
 - Uses the **Browser History API**
 - Matches URL with configured routes
 - Renders the corresponding component
-

5 Why does page not reload in React Router?

Answer:

Because React Router changes the URL using the **History API** (`pushState`) instead of making a server request.

6 What is `<BrowserRouter>` ?

Answer:

`BrowserRouter` enables routing by using the **HTML5 History API**.

Without it, React Router **will not work**.

7 Why should the entire app be wrapped in BrowserRouter?

Answer:

So that **all components** can access routing features like:

- Routes
- Link
- useNavigate
- useParams

8 What is the difference between `Routes` and `Route` ?

Answer:

- `Routes` → container for route definitions
- `Route` → maps a URL path to a component

9 What does `path` and `element` mean in `Route`?

Answer:

- `path` → URL pattern
- `element` → component to render when path matches

10 Difference between `<a>` tag and `<Link>` ?

Answer:

Anchor <code><a></code>	<code><Link></code>
Full page reload	No reload
Server request	Client-side
Slower	Faster
Not SPA-friendly	SPA-friendly

11 Why should we avoid `<a>` tag in React apps?

Answer:

Because it causes **full page reload**, breaking SPA behavior.

1 2 What is JSX navigation?

Answer:

Navigation done using `<Link>` inside JSX.

1 3 What is programmatic navigation?

Answer:

Navigation triggered using JavaScript logic instead of clicking a link (e.g., button click, API success).

1 4 Which hook is used for programmatic navigation?

Answer:

`useNavigate`

1 5 Give real-world use cases of `useNavigate`

Answer:

- Navigate after form submission
 - Navigate after API success
 - Redirect after login/logout
 - Navigate to product details page
-

1 6 What is navigation state?

Answer:

Temporary data passed during navigation that:

- Is **not visible in URL**
 - Is lost on page refresh
-

1 7 How do you pass data using navigation state?

Answer:

```
navigate("/path", {state: data });
```

1 8 How do you access navigation state?

Answer:

Using `useLocation()`.

1 9 Difference between navigation state and URL params?

Answer:

Navigation State	URL Params
Not in URL	Visible in URL
Temporary	Persistent
Lost on refresh	Survives refresh
Not shareable	Shareable

2 0 What is dynamic routing?

Answer:

Routing where parts of the URL act as variables.

```
/ProductDetails/:id
```

2 1 How do you access URL parameters?

Answer:

Using `useParams()`.

2 2 Can we send multiple parameters in URL?

Answer:

Yes, by defining multiple dynamic segments.

2 3 When should you use URL params over navigation state?

Answer:

- When data must persist on refresh
- When URL should be shareable

- When fetching data from API

2 4 What is wildcard routing?

Answer:

Used to catch unmatched routes and show a **404 page**.

```
path="*"
```

2 5 Why is 404 route important?

Answer:

To handle invalid URLs and improve user experience.

◆ LAYOUT & NESTED ROUTES

2 6 What are layout routes?

Answer:

Routes that define **common UI** (header/footer/sidebar) shared across multiple pages.

2 7 What problem do layout routes solve?

Answer:

Avoid duplication of common components across pages.

2 8 What is `<Outlet>` ?

Answer:

A placeholder where **nested route components** render.

2 9 What is the purpose of `index` route?

Answer:

It defines the **default child route**.

3 0 What are protected routes?

Answer:

Routes accessible only to **authenticated users**.

3 1 How do you implement protected routes in React Router?

Answer:

By creating a wrapper component that checks authentication and conditionally renders children or redirects to login.

Major Assignment:



BILLING SYSTEM – FUNCTIONAL REQUIREMENTS (DETAILED)

This application simulates a **real-world billing / POS / checkout system** used in retail stores, restaurants, and ecommerce platforms.

1 Authentication (Fake Authentication)



Goal

Restrict access to billing pages unless the user is logged in.



Functional Requirements

- The application must have a **Login Page**.
- User enters:
 - Username
 - Password (can be dummy).
- On clicking **Login**:
 - Set a login flag (`isAuthenticated = true`)

- Redirect the user to **Products Page**.
 - If the user is **not authenticated**:
 - Any attempt to access:
 - `/products`
 - `/cart`
 - `/invoice/:invoiceId`
 - Should automatically redirect to `/login`.
-

2 Products Page

Functional Requirements

- Fetch or load a **list of products**:
 - id
 - name
 - price
 - category (optional)
 - Display products in:
 - Cards or list UI.
 - Each product must have:
 - **Add to Cart** button.
 - On clicking **Add to Cart**:
 - If product does not exist → add with quantity `1`
 - If already exists → increase quantity by `1`
-

3 Cart Page

Functional Requirements

- Display all items added to cart.
- For each cart item:

- Product name
- Price
- Quantity
- Total price (price × qty)
- Provide actions:
 -  Increase quantity
 -  Decrease quantity
 -  Remove item completely
- Quantity should **never go below 1**.
- If quantity becomes , item should be removed automatically.

Summary Section (Mandatory)

Show:

- **Total Quantity**
- **Total Price (before tax & discount)**

Billing Rules (Business Logic)

Functional Requirements

◆ GST Calculation

- GST is **18%** of total price.
- Formula:

```
GST = totalPrice × 0.18
```

◆ Discount Rules

Total Price	Discount
> ₹5,000	10%
> ₹10,000	20%

- Discount applies **only on total price**.
- Only **one discount slab** should apply.

◆ Final Amount Calculation

```
Final Amount = Total Price + GST - Discount
```



Cart Summary Must Show

- Total Price
- GST amount
- Discount amount
- Final Payable Amount

5 Invoice Page



Functional Requirements

- On clicking **Checkout**:
 - Navigate to:

```
/invoice/:invoiceId
```

- `invoiceId` should be:
 - Timestamp OR
 - UUID OR
 - Random number
- Invoice page must receive data using:
 - `navigate(path, { state })`
- **Do NOT refetch cart data** on invoice page.



Invoice Page Must Display

- Invoice ID
- Date & Time
- List of purchased items
- Quantity and price
- Total price
- GST
- Discount
- Final paid amount

Edge Cases (Bonus Points)

- Cart empty → Disable checkout
 - Refresh invoice page → Show warning message
 - Unauthorized access → Redirect to login
 - Invalid route → Show 404 page
-

UseReducer

- Its one of the way to store component state similar to usestate.
- When ever usestate is becoming complex and if you have multiple usestates inside a component then you can use useReducer.
- It feels like redux inside a component(It has action, reducer & dispatch but inside the component)

When to use useReducer ?

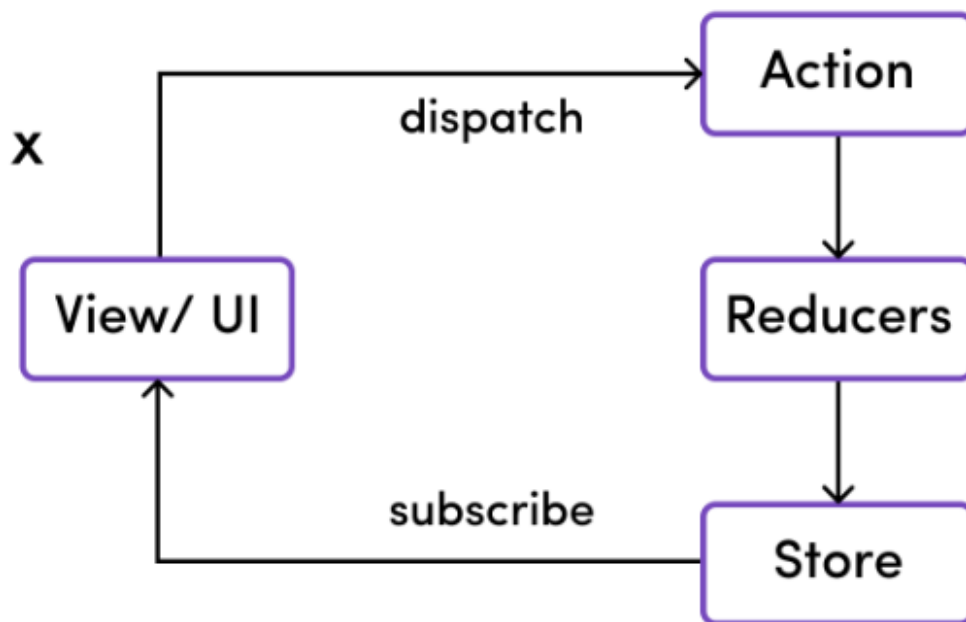
- To mange complex state in a component.
- When next state depends on previous state.
 - Examples:
 - Incrementing or decrementing quantity

- Adding or removing item from cart
- If state is dependent on other state
 - Increment based on step value entered in textbox
- Updating multiple properties at once
 - Examples:
 - Updating quantity and total price at once.

What are the main parts of useReducer ?

1. Reducer function
2. state
3. dispatch function
4. initial state

Visual diagram:



initialState:

- It will be the starting state of your component.

reducer function:

- It will take the existing state and action payload as parameters and performs action on existing state and it will return a new state.

action:

- How this usereducer will know which state value you are updating ? It will be the combination of multiple states right ? whether you are incrementing/decrementing /calculating total price etc. How this reducer knows this ? by using type inside action payload.
- It is an object which contains type property which specifies what action user performed.It can be increment action/decrement action/reset action etc.

dispatch:

- It is a function which is used to send user action to the reducer.

Example Explanation:

First create increment & decrement functionality using useState and then showcase this below example.

```
import { useReducer } from "react";

const initialState = {
  count: 0,
};

const reducer = (state, action) => {
  switch (action.type) {
    case "increment":
      return { count: state.count + 1 };
    case "decrement":
      return { count: state.count - 1 };
    default:
      return state;
  }
};
```

```

const Parent = () => {
  const [state, dispatch] = useReducer(reducer, initialState);

  return (
    <>
      <button onClick={() => dispatch({ type: "decrement"
    })}>-</button>
      <h1>{state.count}</h1>
      <button onClick={() => dispatch({ type: "increment"
    })}>+</button>
    </>
  );
};

export default Parent;

```

Example 2:

Increment & decrement by step value provided by user in textbox.

```

import { useReducer } from "react";

const initialState = {
  count: 0,
  step: 0,
};

const reducer = (state, action) => {
  switch (action.type) {
    case "increment":
      return { ...state, count: state.count + state.step };
    case "decrement":
      return { ...state, count: state.count - state.step };
    case "step":
      return { ...state, step: action.step };
  }
};

```

```

    default:
      return state;
    }
  };

  const Parent = () => {
    const [state, dispatch] = useReducer(reducer, initialState);

    return (
      <>
        <input
          onChange={(e) =>
            dispatch({ type: "step", step: Number(e.target.value) })
          }
        />
        <button onClick={() => dispatch({ type: "decrement" })}>-</button>
        <h1>{state.count}</h1>
        <button onClick={() => dispatch({ type: "increment" })}>+</button>
      </>
    );
  };

  export default Parent;

```

Interview Questions & Answers:

1. What is `useReducer` in React?

Answer:

`useReducer` is a React Hook used to manage **complex component state**.

It works similar to Redux by using a **reducer function**, **actions**, and **dispatch**, but it is limited to a single component or component tree.

2. How is `useReducer` different from `useState` ?

Answer:

<code>useState</code>	<code>useReducer</code>
Best for simple state	Best for complex state
Direct state update	State updated via reducer
Hard to manage multiple states	Centralized state logic
Less predictable for complex logic	More predictable

3. When should we use `useReducer` instead of `useState` ?

Answer:

Use `useReducer` when:

- State logic is complex
 - Next state depends on previous state
 - Multiple state values update together
 - State is dependent on other state values
 - You want predictable state transitions
-

4. What are the main parts of `useReducer` ?

Answer:

1. **Reducer function**
 2. **State**
 3. **Dispatch function**
 4. **Initial state**
-

5. What is a reducer function?

Answer:

A reducer is a **pure function** that:

- Takes current `state` and `action`
- Returns a **new state**
- Does not mutate existing state

```
(state, action) => newState
```

6. What is an action in `useReducer` ?

Answer:

An action is an object that describes **what happened**.

```
{type: "increment" }
```

It usually contains:

- `type` → action name
- `payload` → data needed for update

7. What is `dispatch` ?

Answer:

`dispatch` is a function used to **send an action to the reducer**, which then updates the state.

```
dispatch({type: "increment" })
```

- Useful when passing `dispatch` to child components

8. Can we pass `dispatch` to child components?

Answer:

✓ Yes.

This avoids prop drilling state setters.

```
<Child dispatch={dispatch} />
```

Major Assignment:

Problem Statement

Build a **Cart & Billing Module** for an e-commerce application using `useReducer`.

Requirements

Product List

Display products with:

- Name
 - Price
 - **Add to Cart** button
-

Cart Features (useReducer only)

- Add item to cart
 - Increase quantity
 - Decrease quantity
 - Remove item
 - Calculate:
 - Total Quantity
 - Total Price
 - Discount
 - Final Payable Amount
-

Billing Rules

- If totalPrice > ₹5000 → **10% discount**
 - If totalPrice > ₹10000 → **20% discount**
-

Constraints (Strict)

-  No `useState` for cart logic
-  No calculations inside JSX
-  No direct state mutation
-  All logic must be inside reducer



E-Commerce Cart

Products

Laptop – ₹50000

Add to Cart

Headphones – ₹2000

Add to Cart

Keyboard – ₹1500

Add to Cart

Cart

Your cart is empty

Billing Summary

Total Quantity	0
Total Price	₹0
Discount	₹0
Final Amount	₹0

Code splitting & Lazy loading:

Code Splitting

Code splitting means breaking a **large JavaScript bundle** into **smaller chunks** so that the browser doesn't need to download everything at once.

Lazy Loading

Lazy loading means **loading code only when it is required**, instead of loading all pages during the initial application load.

Lazy loading is one of the most common techniques used to achieve code splitting in React.

Why do we need Code Splitting & Lazy Loading?

- If your application is **small**, code splitting and lazy loading are **not mandatory**.
- If your application is **large**, they are **very important** to improve performance.

Example: E-commerce Application

Let's say we have an e-commerce application with:

- Home page
- Products page
- Orders page
- Cart page

✗ Without Lazy Loading

- Entire JavaScript bundle is loaded at once:
 - Home
 - Products
 - Orders
 - Cart

- Even if the user opens **only the Home page**, the browser still downloads **all pages' code**
- This increases:
 - Initial load time
 - Bundle size
 - Time to interactive

✅ With Lazy Loading

- Initially, **only Home page code** is loaded
 - When the user navigates:
 - Products → Products code loads
 - Orders → Orders code loads
 - Other pages are downloaded **only when required**
-

Benefits of Lazy Loading

- 🚀 Faster initial load time
 - 😊 Better user experience
 - ⚡ Improved performance
 - 📦 Smaller initial JavaScript bundle
-

Important: Which Pages Should Be Lazy Loaded?

❌ Do NOT Lazy Load

- Landing pages
- Home page
- Pages required immediately after app load

✅ Lazy Load

- Heavy pages (Products, Orders, Reports)
- Pages that are **not the landing page**

- Pages that are **rarely used**

Example Code:

Important Rule

| Lazy-loaded components **MUST** have a default export

Home Component

```
export const Home = () => {  
  return <h1>Home</h1>;  
};
```

Products Component (Default Export Required)

```
const Products = () => {  
  return <h1>Products</h1>;  
};  
  
export default Products;
```

App.jsx – Lazy Loading with React Router

```
import { lazy, Suspense } from "react";  
import { BrowserRouter, Routes, Route, Link } from "react-router-dom";  
import { Home } from "../Components/Home";  
  
const Products = lazy(() => import("../Components/Products"));  
  
function App() {  
  return (  
    <BrowserRouter>  
      <nav>
```

```

    <Link to="/">Home</Link>
    <Link to="/Products">Products</Link>
  </nav>

  <Suspense fallback={<h3>Loading page...</h3>}>
    <Routes>
      <Route path="/" element={<Home />} />
      <Route path="/Products" element={<Produ
cts />} />
    </Routes>
  </Suspense>
</BrowserRouter>
);
}

export default App;

```

Suspense:

`Suspense` shows a **fallback UI** while a lazy-loaded component is being downloaded.

```

<Suspense fallback={<h3>Loading page...</h3>}>
  <Products />
</Suspense>

```

Simulating Delay

To **visually demonstrate lazy loading**, we can artificially delay loading:

```

export function delay(ms) {
  return new Promise((resolve) => setTimeout(resolve, m
s));
}

const Products = lazy(() =>

```



```
delay(2000).then(() =>import("./Components/Products"))  
);
```

Now the fallback UI will be visible for **2 seconds**, clearly showing lazy loading in action.

Interview questions and answers:

1. What is code splitting?

Answer:

Code splitting is the process of breaking a large JavaScript bundle into smaller chunks so that the browser can load only the required code instead of downloading everything at once.

2. What is lazy loading in React?

Answer:

Lazy loading is a technique where components are loaded **only when they are needed**, instead of loading all components during the initial application load.

3. Why do we need lazy loading?

Answer:

Lazy loading improves:

- Initial page load time
- Application performance
- User experience

Especially useful for large applications.

4. Is code splitting and lazy loading required for small applications?

Answer:

No. For small applications, the performance gain is minimal. These techniques are mainly beneficial for large applications.

5. How is lazy loading implemented in React?

Answer:

Using:

- `React.lazy()` for dynamic import
 - `Suspense` to show fallback UI while loading
-

6. What is `React.lazy()` ?

Answer:

`React.lazy()` allows you to dynamically import a component and load it only when it is rendered.

```
const Products = lazy(() => import("./Products"));
```

7. Why is `Suspense` required?

Answer:

`Suspense` is required to display a fallback UI while a lazy-loaded component is being downloaded.

8. What happens if we use `React.lazy()` without `Suspense` ?

Answer:

React will throw an error because it doesn't know what to render while the component is loading.

9. Can we lazy load multiple components?

Answer:

Yes. Any component can be lazy loaded as long as it has a default export and is wrapped inside `Suspense`.

10. Why must lazy-loaded components have a default export?

Answer:

`React.lazy()` works with dynamic imports, and React expects the imported module to resolve to a default export.

11. Which pages should NOT be lazy loaded?

Answer:

- Landing pages
 - Home page
 - Pages needed immediately after app load
-

12. Which pages are ideal for lazy loading?

Answer:

- Heavy pages (Products, Orders, Reports)
 - Rarely used pages
 - Pages not required on initial load
-

13. How does lazy loading affect navigation?

Answer:

- First navigation to a lazy-loaded page may be slightly slower
 - Subsequent navigations are fast because the code is cached
-

14. How can you verify lazy loading in the browser?

Answer:

Using the **Network tab**:

- On initial load, only main bundle loads
 - On navigation, additional JS chunks appear
-

Mini Assignments:

1. Real E-Commerce Simulation

Objective: Simulate real-world performance optimization

Task:

Create pages:

- Home (non-lazy)
- Products (lazy)
- Orders (lazy)
- Admin (lazy)

Rules:

- Home loads instantly
 - Other pages load on navigation
 - Add meaningful fallback UI per page
-

2. Custom Loader UI

- Create a reusable Loader component
 - Use it as fallback in `Suspense`
-

React.memo

What is the problem?

In React:

- When a **parent component re-renders**
- **All its child components re-render by default**
- This happens **even if the child's props have not changed**

This can cause **unnecessary re-renders** and **performance issues**, especially in large applications.

What is React.memo?

- `React.memo` is a **Higher Order Component (HOC)**
- It **stores the previous props** of a component

- On the next render, it **compares old props with new props**
 - If props **have not changed**, React **skips re-rendering** the component
-

Example Scenario

- A **Parent** component has a `count` state
 - Clicking a button updates `count`
 - A **Child** component displays a **static product list**
 - Product list never changes
 - Still, child re-renders whenever parent re-renders ❌
-

Step 1: WITHOUT React.memo (Unnecessary Re-renders)

Parent Component

```
import Child from "./Child";
import { useState } from "react";

// Defined outside so reference does not change
const products = ["Burger", "Fries", "Coke"];

function Parent() {
  const [count, setCount] = useState(0);

  return (
    <>
      <button onClick={() => setCount(count + 1)}>
        Increase
      </button>
      <Child products={products} />
    </>
  );
}
```

```
export default Parent;
```

Child Component (Without React.memo)

```
const Child = ({ products }) => {  
  console.log("ProductList rendered");  
  return <div>{products.length} products</div>;  
};  
  
export default Child;
```

What happens?

- Clicking **Increase** updates parent state
- Parent re-renders 
- Child also re-renders 
- Console log appears **on every click**

```
ProductList rendered
```

 This happens even though `products` never change.

Step 2: WITH React.memo (Re-renders Stopped)

Child Component (With React.memo)

```
import React from "react";  
  
const Child = React.memo(function ProductList({ products })  
{  
  console.log("ProductList rendered");  
  return <div>{products.length} products</div>;  
});
```

```
export default Child;
```

What happens now?

- Child renders **only once** on initial load
- Clicking **Increase** re-renders parent
- `products` prop remains the same
- React.memo compares previous and current props
- Child re-render is **skipped** ✓

📌 Console output:

```
ProductList rendered    (only once)
```

When React.memo REALLY helps ✓

Use React.memo when:

- ✓ Child component is **heavy**
- ✓ Child renders **large lists**
- ✓ Parent re-renders **frequently**
- ✓ Props remain **mostly unchanged**

When NOT to use React.memo

Avoid React.memo when:

- ✗ Component is small
- ✗ Rendering is cheap
- ✗ Props change frequently
- ✗ Application is simple

Overusing React.memo can:

- Increase code complexity
 - Make debugging harder
 - Provide no real performance benefit
-

Interview questions & answers

1. What is React.memo?

Answer:

`React.memo` is a Higher Order Component that prevents unnecessary re-renders of a functional component by **memoizing its props**.

If the props do not change, React skips re-rendering the component.

2. Does React.memo stop re-render when parent re-renders?

Answer:

Yes, **only if the child props remain the same**.

If props change (or reference changes), the child will still re-render.

3. What is the difference between React.memo and useMemo?

Answer:

React.memo	useMemo
Memoizes a component	Memoizes a value
Prevents child re-renders	Prevents expensive recalculations
Used on components	Used inside components

4. Will React.memo prevent re-render if child has its own state?

Answer:

No.

If the **child's own state changes**, it will re-render regardless of React.memo.

5. When should you NOT use React.memo?

Answer:

- Small or simple components
 - Components with frequently changing props
 - Cheap rendering logic
 - When performance issue is not proven
-

6. Is React.memo a performance optimization or a feature?

Answer:

It is a **performance optimization**.

It should be used **only when necessary**, not by default.

7. Can React.memo cause performance issues?

Answer:

Yes.

- Extra memory usage
- Extra prop comparison cost
- Increased complexity

Sometimes re-rendering is **cheaper than memoization**.

8. Real-time scenario where React.memo is useful?

Answer:

- Product list with 1000+ items
- Dashboard cards

- Charts
 - Table rows
 - Reusable UI components in frequently re-rendering parents
-

9. React.memo vs shouldComponentUpdate?

Answer:

`shouldComponentUpdate` is for **class components**.

React.memo provides similar optimization for **functional components**.

10. Is React.memo required for every child component?

Answer:

No.

Only use it when:

- Re-render is expensive
 - Props are stable
 - Performance issue is identified
-

Usememo

`useMemo` in React – Simple & Standard Explanation

What is `useMemo` ?

`useMemo` is a React Hook used to **optimize performance** by **memoizing (remembering)** the result of a calculation between re-renders.

👉 React will **recalculate the value only when its dependencies change**.

👉 Otherwise, it returns the **previously stored value**.

When should you use `useMemo` ?

Use `useMemo` **only when needed**, especially when:

- The calculation is **expensive** (loops, filtering, sorting large data).
- You are creating **large objects or arrays** that are passed to child components.
- Recalculation on every render causes **performance issues**.

❌ Do **not** use `useMemo` for simple or cheap calculations — it adds unnecessary complexity.

Real-World Example (E-commerce)

Imagine an e-commerce website with **1000+ products**.

- User applies filters like:
 - `category`
 - `price range`
 - `brand`
- Filtering 1000+ products on **every render** is slow.

❌ Without `useMemo`

Filtering runs on **every re-render**, even when filters don't change.

✅ With `useMemo`

Filtering runs **only when products or filters change**.

```
const filteredProducts = useMemo(() => {  
  return products.filter(  
    (p) => p.category === selectedCategory  
  );  
}, [products, selectedCategory]);
```

✅ Now React recalculates **only when required**.

Example: Cart Total Price Calculation

❌ Without `useMemo` (recalculates every render)

```

function calculateTotal(items) {
  let total =0;
  for (let i =0; i < items.length; i++) {
    total += items[i].price * items[i].qty;
  }
  return total;
}

export default function App() {

  const items = [
    {id:1,name:"Burger",price:100,qty:2 },
    {id:2,name:"Fries",price:50,qty:3 },
    {id:3,name:"Coke",price:30,qty:1 },
  ];

  const [count, setCount] =useState(0);

  const totalPrice = calculateTotal(items);// runs every render

  return (
    <>
      <button onClick={() => setCount(count + 1)}>
        Re-render
      </button>
      <h3>Total Price: ₹{totalPrice}</h3>
    </>
  );
}

```

● Problem:

- Clicking the button causes re-render.
- `calculateTotal` runs again **even though items didn't change**.

✓ Optimized with `useMemo`

```
import { useMemo } from "react";

const items = [
  {id:1,name:"Burger",price:100,qty:2 },
  {id:2,name:"Fries",price:50,qty:3 },
  {id:3,name:"Coke",price:30,qty:1 },
];

function Cart() {
  const totalPrice = useMemo(() => {
    let total =0;
    for (let i =0; i < items.length; i++) {
      total += items[i].price * items[i].qty;
    }
    return total;
  }, [items]);

  return <h3>Total Price: ₹{totalPrice}</h3>;
}

export default Cart;
```

- ✓ Calculation runs **only once** (or when `items` change).
- ✓ No unnecessary recalculation.
- ✓ Better performance for large carts.

Interview Questions & Answers:

1. What is useMemo and why do we use it?

Answer:

`useMemo` is a React hook used to **memoize a computed value** so that it is **recalculated only when its dependencies change**.

It helps **avoid expensive recalculations** on every render and improves performance.

```
const result =useMemo(() => expensiveCalculation(a, b), [a, b]);
```

2. What happens if we do not pass dependencies to useMemo?

Answer:

-  **No dependency array** → value is recalculated on **every render** (no benefit)
-  **Empty dependency array** `[]` → value is calculated **only once**
-  **With dependencies** → recalculated only when dependencies change

```
useMemo(() => compute(), []); // runs only once
```

3. Difference between useMemo and useCallback?

Answer:

useMemo	useCallback
Memoizes a value	Memoizes a function
Returns computed result	Returns function reference
Used for expensive calculations	Used to prevent function re-creation

```
useMemo(() => value, []);  
useCallback(() => fn(), []);
```

ContextApi:

What is Context API?

Context API is a built-in React feature that allows us to:

- Share data across components
- Avoid props drilling
- Share data between sibling components

Lets say i have fetched user information in users component and i wan to use it in cart component and product details components which are siblings of user component. Here we can create a global state using context and utilize it in all other components.

Folder structure:

```
src/  
├── components/  
│   ├── User.jsx  
│   └── Cart.jsx  
├── context/  
│   ├── UserContext.jsx  
│   └── UserProvider.jsx  
├── services/  
│   └── userService.js  
└── App.jsx
```

Step 1: Create Components

User.jsx

```
export const User = () => {  
  return <h1>User Component</h1>;  
};
```

Cart.jsx

```
export const Cart = () => {  
  return <h1>Cart Component</h1>;  
};
```

Create Fake API Service

services/userService.js

```
export const fetchUserInfoApi = () => {  
  return Promise.resolve([  
    {id:1,name:"Saikrishna" }  
  ]);  
};
```

Step 3: Create Context

context/UserContext.jsx

```
import { createContext } from "react";  
  
export const UserContext = createContext(null);
```

Step 4: Create Provider (IMPORTANT)

Provider is responsible for owning the state

context/UserProvider.jsx

```
import { useState } from "react";  
import { UserContext } from "../UserContext";  
  
export const UserProvider = ({ children }) => {  
  const [user, setUser] = useState([]);  
  
  return (  
    <UserContext.Provider value={{user, setUser}}>  
      {children}  
    </UserContext.Provider>  
  )  
};
```



```
);  
};
```

- ✓ Correct place for state
- ✓ Clean separation of responsibility

Step 5: Wrap Components with Provider

App.jsx

```
import { User } from "../components/User";  
import { Cart } from "../components/Cart";  
import { UserProvider } from "../context/UserProvider";  
  
function App() {  
  return (  
    <UserProvider>  
      <User />  
      <Cart />  
    </UserProvider>  
  );  
}  
  
export default App;
```

Step 6: Fetch Data in User Component & Store in Context

User.jsx

```
import { useEffect, useContext } from "react";  
import { fetchUserInfoApi } from "../services/userService";  
import { UserContext } from "../context/UserContext";  
  
export const User = () => {  
  const { user, setUser } = useContext(UserContext);
```

```

useEffect(() => {
  async function getUser() {
    const data = await fetchUserInfoApi();
    setUser(data);
  }

  getUser();
}, []);

return (
  <h1>
    User Component: {user[0]?.id} {user[0]?.name}
  </h1>
);
};

```

Step 7: Consume Context in Cart Component

Cart.jsx

```

import { useContext } from "react";
import { UserContext } from "../context/UserContext";

export const Cart = () => {
  const { user } = useContext(UserContext);

  return (
    <h1>
      Cart Component: {user[0]?.id} {user[0]?.name}
    </h1>
  );
};

```

Multiple Contexts Example

```
<UserProvider>
  <OrderProvider>
    <ProductProvider>
      <User />
      <Cart />
    </ProductProvider>
  </OrderProvider>
</UserProvider>
```

Each context handles one responsibility

Mini Assignments:

1. Implement CartContext
 2. Implement ThemeContext
-

Interview questions & answers

1. What is Context API in React?

Answer:

Context API is a built-in React feature that allows us to share data across components without passing props at every level.

It is mainly used to avoid **props drilling** and to share **global data** like user info, theme, language, cart data, etc.

2. What problems does Context API solve?

Answer:

Context API solves:

- Props drilling
- Sharing data between sibling components

- Managing global or common application state
-

3. What are the main parts of Context API?

Answer:

Context API has **three main parts**:

1. `createContext()` – creates the context
 2. `Provider` – provides data to components
 3. `Consumer` / `useContext()` – consumes the data
-

4. What is props drilling?

Answer:

Props drilling is the process of passing data from a parent component to deeply nested child components through multiple intermediate components that do not need the data.

5. How do you create a context in React?

Answer:

```
import { createContext } from "react";

export const UserContext = createContext(null);
```

6. What is a Provider in Context API?

Answer:

A Provider is a component that wraps other components and supplies data to them using the `value` prop.

Only components inside the Provider can access the context data.

7. Can we use multiple contexts in one application?

Answer:

Yes. We can use multiple contexts such as `UserContext`, `CartContext`, `OrderContext`, etc.

Using multiple small contexts is better than one large context.

8. When should we NOT use Context API?

Answer:

We should avoid Context API when:

- State updates very frequently
- Performance is critical
- Complex state logic is involved

In such cases, `Redux` or `Zustand` is better.

9. Real-world use cases of Context API?

Answer:

- Authentication state
 - Theme / dark mode
 - Language settings
 - Cart data
 - User profile info
-

Props Drilling:

- Prop drilling means passing data from parent component to deeply nested child component through many intermediate components, even though those components does not need those props.
- Lets say i have a below hierarchy.,
 - `App` $\Rightarrow$ `Layout` $\Rightarrow$ `Header` $\Rightarrow$ `Cart` $\Rightarrow$ `CartHeader`

- Now i want to send the current type selected by user from App component to Cart Header component, then i need to send through this hierarchy levels even though these components dont need the currency type.

Problems caused by props drilling:

- Components becomes tightly coupled as middle components depends on parents.
- Reusability decreases
- Adding new data ⇒ we need to send through everywhere
- Refactoring becomes hard.

For small applications, with 2+ hierarchy levels, its fine but For Huge applications, its not recommended.

To avoid this prop drilling, we use centralized state management like context api/Redux.

Higher Order components:

- A Higher order function is a function which takes component as a input and will return a new component with additional features.
- Mainly used for code reusability.

Example:

Lets say i have some authentication logic in multiple components. I want to make it reusable across the components. For this i will create a HOC and Will move authentication logic to HOC.

Without HOC:

```
function Profile() {  
  const isLoggedIn = true;  
  
  if (!isLoggedIn) {
```

```

    return <p>Please login</p>;
  }

  return <h2>Profile Page</h2>;
}

```

With HOC:

Profile Component:

```

import { WithAuth } from "../HOC/WithAuth";

export const Profile = (props) => {
  return <h1>Iam {props.name}</h1>;
};

export default WithAuth(Profile);

```

```

=====
=====

```

HOC Component:

```

import React from "react";

export const WithAuth = (WrapperComponent) => {
  return function AuthComponent(props) {
    // Here, These props are props which comes to profile component from its parent
    const isLoggedIn = true;

    if (!isLoggedIn) {
      return <h1>Iam loggedout</h1>;
    }
  }
}

```

```

    return <WrapperComponent {...props} />;
  };
};

```

- Lets say i want to add Logger functionality for multiple components, then i will create a HOC for this.
- If we want to add multiple reusable features to a component, then i use composition of HOC's
- Here i have created WithAuth for authentication and WithLogger for logging. and i have applied these HOC's yto profile component.

WithAuth HOC :

```

export const WithAuth = (WrapperComponent) => {
  return function AuthComponent(props) {
    const isLoggedIn = true;

    if (!isLoggedIn) {
      return <h1>Iam loggedout</h1>;
    }

    return <WrapperComponent {...props} />;
  };
};

```

WithLoggerHOC :

```

export const WithLogger = (WrapperComponent) => {
  return function LoggerComponent(props) {
    console.log(`Logger component ${props.name}`);

    return <WrapperComponent {...props} />;
  };
};

```


Profile Component :

```
import { WithAuth } from "../HOC/WithAuth";
import { WithLogger } from "../HOC/WithLogger";

export const Profile = (props) => {
  return <h1>Iam {props.name}</h1>;
};

export default WithAuth(WithLogger(Profile));
```

Mini Assignment:

- Cart, Checkout, Orders pages should be accessible **only to logged-in users**.
 - Create a loading indicator HOC while fetching products from API's
-

Interview Questions & Answers:

1. What is a Higher-Order Component (HOC)?

Answer:

A Higher-Order Component is a **function that takes a component as input and returns a new enhanced component**.

```
const withEnhancement = (WrappedComponent) => {
  return functionEnhancedComponent(props) {
    return <WrappedComponent {...props} />;
  };
};
```

2. Why do we use HOCs?

Answer:

We use HOCs to **reuse component logic that affects rendering or UI behavior** such as:

- Authentication
 - Authorization
 - Logging
 - Error boundaries
 - Loading states
-

3. Are HOCs still used after Hooks?

Answer:

Yes, but less frequently.

Hooks replaced HOCs for **logic reuse**, but HOCs are still useful for:

- Class components
 - Library-level code
-

4. Can we use multiple HOCs on a single component?

Answer:

Yes. This is called **HOC composition**.

```
withAuth(withLogger(Products))
```

Order matters and affects execution flow.

5. Why does the order of HOCs matter?

Answer:

Because HOCs are functions and are executed **inside-out**.

```
withAuth(withLogger(Component))
```

- `withLogger` runs first
- `withAuth` runs next

6. Do HOCs modify the original component?

Answer:

No.

HOCs follow **immutability** — they return a **new component**.

7. What are disadvantages of HOCs?

Answer:

- Wrapper hell
- Harder debugging
- Extra component nesting

8. Can we pass props through HOCs?

Answer:

Yes, using prop forwarding:

```
return <WrappedComponent {...props} />;
```

9. Can HOCs be replaced with JSX composition?

Answer:

Yes, in many cases.

Modern React prefers **composition over inheritance**.

10. Give a real-world HOC example

Answer:

Authentication, role-based access, logging, error boundaries.

Class components:

Class components are the **old way** of creating components in React using JavaScript classes. They were the primary way before **React Hooks** (introduced in React 16.8, 2019).

```
import { Component } from "react";

class Header extends Component {
  render() {
    return <h1>Class Header</h1>;
  }
}

export default Header;
```

Why we need to extend from `Component` ?

- Because it Gives access to:
 - `this.props`
 - `this.state`
 - `this.setState()`
 - lifecycle methods

Without extending `Component`, our component does not gets these features.

Constructor:

```
constructor(props) {
  super(props);
  this.state = { count: 0 };
}
```

- Called **once** when the component instance is created
- It is used for:
 - Initializing the state
 - Binding methods (old pattern)

constructor runs **before mounting**, during component creation

Super():

In React class components, `super(props)` is needed because when you extend `React.Component`, you must call the parent class's constructor before using `this` in your constructor. Calling `super(props)` does two things: first, it satisfies JavaScript's requirement that you call the parent constructor before accessing `this`, and second, it passes `props` to `React.Component` so that React can set up `this.props` correctly inside your constructor. Without `super()`, you'll get an error when trying to use `this.state` or any `this` keyword, and without passing `props` to it, `this.props` will be undefined specifically inside the constructor (though it works fine everywhere else because React sets it up after construction).

```
class MyComponent extends React.Component {
  constructor(props) {
    super(props) // Calls React.Component's constructor with props
    this.state = { count: 0 } // Now 'this' works
    console.log(this.props.name) // Now 'this.props' works in constructor
  }
}
```

Modern React (16+)

BUT - you can avoid `super()` by not using a constructor at all:

```
// Modern class component WITHOUT constructor
class MyComponent extends React.Component {
  state = { count: 0 } // Class field syntax - no constructor needed!
```

```
handleClick = () => { // Arrow function - auto-binds 'this'  
  this.setState({ count: this.state.count + 1 })  
}  
  
render() {  
  return <div onClick={this.handleClick}>{this.state.count}</div>  
}
```

this Keyword

- Refers to the **current component instance**
- Used to access:
 - state → `this.state`
 - props → `this.props`
 - methods → `this.methodName`

State & setState:

State is **mutable data** that belongs to a component.

```
this.state = { count: 0 };
```

Updating state (WRONG)

```
this.state.count = 1; // ❌
```

Updating state (CORRECT)

```
this.setState({ count: 1 });
```

Functional State Update

Use this when new state depends on old state.

```
this.setState(prevState => ({  
  count: prevState.count + 1  
}));
```

Event Handling in React – Class Components

What is Event Handling?

Event handling means **responding to user actions** like:

- click
- input change
- submit
- mouse hover

Examples:

- Button click
- Typing in input box
- Form submission

How Events Work in React (Important Concept)

React uses **Synthetic Events**:

- Cross-browser wrapper around native events
- Same event behavior in all browsers
- Automatically managed by React

Example:

```
handleClick(event) {  
  console.log(event);
```

```
}
```

Basic Event Handling Example (Class Component)

```
import React, {Component } from "react";

class ClickExample extends Component {
  handleClick() {
    console.log("Button clicked");
  }

  render() {
    return (
      <button onClick={this.handleClick}>
        Click Me
      </button>
    );
  }
}

export default ClickExample;
```

✗ This will throw an error because this is undefined in this case.

Why **this** Binding is Required

In JavaScript classes:

- Methods do NOT automatically bind **this**
- When method is used as a callback, **this** is lost

So React doesn't know:

```
this.handleClick
```

Ways to Handle Events Correctly (VERY IMPORTANT)

Method 1: Bind in Constructor (Most Interview-Friendly)

```
class ClickExample extends Component {
  constructor(props) {
    super(props);
    this.handleClick = this.handleClick.bind(this);
  }

  handleClick() {
    console.log("Button clicked");
  }

  render() {
    return (
      <button onClick={this.handleClick}>
        Click Me
      </button>
    );
  }
}
```

Method 2: Arrow Function Class Property (Most Used)

```
class ClickExample extends Component {
  handleClick = () => {
    console.log("Button clicked");
  };

  render() {
    return (
      <button onClick={this.handleClick}>
        Click Me
      </button>
    );
  }
}
```

```
}  
}
```

Here, `this` is auto-bound when we use arrow function syntax

Props in Class Components

Props (short for "properties") are read-only inputs passed from a parent component to a child component in React. They allow you to pass data and configuration down the component tree.

Key Characteristics

In class components, props are accessed via `this.props` and have these important features:

- **Read-only:** You cannot modify props inside the component that receives them
- **Passed from parent:** Props flow downward from parent to child components
- **Any data type:** Props can be strings, numbers, objects, arrays, functions, or even other React components

Example

Here's a practical example showing how props work in class components:

```
import React, { Component } from 'react';  
  
// Parent Component  
class App extends Component {  
  render() {  
    return (  
      <div>  
        <UserCard  
          name="Alice Johnson"  
          age={28}  
        />  
      </div>  
    );  
  }  
}
```

```

        role="Software Engineer"
        isActive={true}
      />
      <UserCard
        name="Bob Smith"
        age={35}
        role="Product Manager"
        isActive={false}
      />
    </div>
  );
}
}

// Child Component
class UserCard extends Component {
  render() {
    const { name, age, role, isActive } = this.props;

    return (
      <div className="user-card">
        <h2>{name}</h2>
        <p>Age: {age}</p>
        <p>Role: {role}</p>
        <p>Status: {isActive ? 'Active' : 'Inactive'}</p>
      </div>
    );
  }
}

export default App;

```

In this example, the `App` component passes four props (`name`, `age`, `role`, `isActive`) to the `UserCard` component, which then accesses them using `this.props` and displays the information. Each `UserCard` instance receives different prop values, making the component reusable.

Passing Functions as Props

Passing functions as props allows a child component to communicate back to its parent component. This is a common pattern in React for handling events and updating state in the parent based on actions in the child.

Why Pass Functions?

Since props flow downward (parent → child) and are read-only, children can't directly modify parent state. By passing a function as a prop, the child can call that function to trigger changes in the parent component.

Example

Here's a practical example of a parent component passing a function to a child:

```
import React, { Component } from 'react';

// Parent Component
class ShoppingCart extends Component {
  constructor(props) {
    super(props);
    this.state = {
      items: [],
      totalPrice: 0
    };
  }

  // Function to be passed as prop
  addItem = (itemName, price) => {
    this.setState(prevState => ({
      items: [...prevState.items, itemName],
      totalPrice: prevState.totalPrice + price
    }));
  }

  render() {
    return (
      <div>
```

```

    <h1>Shopping Cart</h1>
    <p>Total Items: {this.state.items.length}</p>
    <p>Total Price: ${this.state.totalPrice}</p>

    { /* Passing function as prop */ }
    <ProductList onAddItem={this.addItem} />
  </div>
);
}
}

// Child Component
class ProductList extends Component {
  handleAddToCart = (name, price) => {
    // Calling the parent's function via props
    this.props.onAddItem(name, price);
  }

  render() {
    return (
      <div>
        <h2>Available Products</h2>
        <button onClick={() => this.handleAddToCart('Laptop', 999)}>
          Add Laptop ($999)
        </button>
        <button onClick={() => this.handleAddToCart('Mouse', 25)}>
          Add Mouse ($25)
        </button>
        <button onClick={() => this.handleAddToCart('Keyboard', 75)}>
          Add Keyboard ($75)
        </button>
      </div>
    );
  }
}

```

```
export default ShoppingCart;
```

How It Works

1. The parent (`ShoppingCart`) defines the `addItem` function and manages the state
2. The parent passes `addItem` as a prop called `onAddItem` to the child (`ProductList`)
3. When a button is clicked in the child, it calls `this.props.onAddItem(name, price)`
4. This executes the parent's `addItem` function, updating the parent's state
5. The parent re-renders with the updated cart information

This pattern enables **two-way communication**: data flows down via props, and events flow up via function calls.

Default Props

Default props allow you to specify default values for props in case the parent component doesn't provide them. This is useful for making components more flexible and preventing errors when props are missing.

Setting Default Props

In class components, you can set default props using the static property

```
defaultProps .
```

Example

Here's a practical example showing how default props work:

```
import React, { Component } from 'react';

// Child Component with Default Props
class Button extends Component {
  // Setting default props
  static defaultProps = {
    text: 'Click Me',
```

```

    color: 'blue',
    size: 'medium',
    disabled: false
  };

  render() {
    const { text, color, size, disabled } = this.props;

    const buttonStyle = {
      backgroundColor: color,
      padding: size === 'large' ? '15px 30px' : size === 'small' ? '5px 10px' : '10px 20px',
      fontSize: size === 'large' ? '18px' : size === 'small' ? '12px' : '14px',
      border: 'none',
      borderRadius: '5px',
      color: 'white',
      cursor: disabled ? 'not-allowed' : 'pointer',
      opacity: disabled ? 0.5 : 1
    };

    return (
      <button style={buttonStyle} disabled={disabled}>
        {text}
      </button>
    );
  }
}

// Parent Component
class App extends Component {
  render() {
    return (
      <div>
        <h1>Button Examples</h1>

        {/* Using all default props */}
        <Button />
      </div>
    );
  }
}

```

```

    {/* Overriding some default props */}
    <Button text="Submit" color="green" />

    {/* Overriding all default props */}
    <Button
      text="Delete"
      color="red"
      size="large"
      disabled={false}
    />

    {/* Using default text and color, custom size */}
    <Button size="small" />
  </div>
);
}
}

export default App;

```

Alternative Syntax

You can also set default props outside the class:

```

class Button extends Component {
  render() {
    const { text, color } = this.props;
    return <button style={{ backgroundColor: color }}>{text}</button>;
  }
}

// Setting default props outside the class
Button.defaultProps = {
  text: 'Click Me',

```



```
    color: 'blue'  
  };
```

How It Works

- If a prop is **not provided** by the parent, React uses the value from `defaultProps`
- If a prop **is provided** by the parent, it overrides the default value
- This makes components more robust and reduces the need for conditional checks

In the example above, the first `<Button />` uses all default values, while the others override specific props as needed.

React Class Component Lifecycle Phases

React class components go through different **lifecycle phases** from creation to removal. Each phase has specific methods (lifecycle methods) that you can use to run code at particular times.

The Three Main Phases

1. Mounting Phase (Birth)

When a component is being created and inserted into the DOM for the first time.

2. Updating Phase (Growth)

When a component is being re-rendered due to changes in props or state.

3. Unmounting Phase (Death)

When a component is being removed from the DOM.

1. Mounting Phase

These methods are called in order when a component is being created:

a) `constructor()`

- Called before the component is mounted
- Used for initializing state and binding methods
- Must call `super(props)` first

b) `render()`

- The only required method in a class component
- Returns JSX to be rendered
- Should be a pure function (no side effects)

c) `componentDidMount()`

- Called immediately after the component is mounted
- Perfect for API calls, subscriptions, or DOM manipulation
- Component and DOM are now available

`Constructor → render → componentDidMount`

2. Updating Phase

These methods are called when props or state change:

a) `shouldComponentUpdate(nextProps, nextState)`

- Determines if component should re-render
- Returns true or false
- Used for performance optimization

b) `render()`

- Re-renders the component with updated data

c) `componentDidUpdate(prevProps, prevState, snapshot)`

- Called after the component updates
- Good for DOM operations or API calls based on prop/state changes
- Can compare previous and current props/state

`shouldComponentUpdate → render → componentDidUpdate`

3. Unmounting Phase

`componentWillUnmount()`

- Called right before the component is removed from the DOM
- Used for cleanup: clearing timers, canceling API requests, removing event listeners
- This is your last chance to perform cleanup

Complete Example

Here's a comprehensive example showing all major lifecycle methods:

```
import React, { Component } from 'react';

class LifecycleDemo extends Component {
  constructor(props) {
    super(props);
    console.log('1. Constructor - Component is being created');
    this.state = {
      count: 0,
      data: null
    };
  }

  componentDidMount() {
    console.log('4. componentDidMount - Component mounted to DOM');
    // API calls, subscriptions, timers
    this.timer = setInterval(() => {
      console.log('Timer running...');
    }, 2000);

    // Simulating API call
    fetch('https://api.example.com/data')
      .then(response => response.json())
      .then(data => this.setState({ data }))
      .catch(error => console.log(error));
  }

  shouldComponentUpdate(nextProps, nextState) {
```

```

    console.log('5. shouldComponentUpdate - Should component re-render?');
    // Return true to allow update, false to prevent
    return true;
}

componentDidUpdate(prevProps, prevState, snapshot) {
    console.log('7. componentDidUpdate - Component updated');
    console.log('Previous count:', prevState.count);
    console.log('Current count:', this.state.count);
    if (snapshot) {
        console.log('Snapshot:', snapshot);
    }
}

componentWillUnmount() {
    console.log('8. componentWillUnmount - Component is being removed');
    // Cleanup
    clearInterval(this.timer);
}

render() {
    console.log('3. Render - Rendering component');
    return (
        <div>
            <h2>Lifecycle Demo</h2>
            <p>Count: {this.state.count}</p>
            <button onClick={() => this.setState({ count: this.state.count + 1 })}>
                Increment
            </button>
        </div>
    );
}
}

```

```
// Parent component to demonstrate mounting/unmounting
class App extends Component {
  state = { showComponent: true };

  render() {
    return (
      <div>
        <button onClick={() => this.setState({ showComponent: !this.state.showComponent })}>
          {this.state.showComponent ? 'Unmount' : 'Mount'}
        </button>
        {this.state.showComponent && <LifecycleDemo />}
      </div>
    );
  }
}

export default App;
```

Common Use Cases

Lifecycle Method	Common Use Cases
<code>constructor()</code>	Initialize state, bind methods
<code>componentDidMount()</code>	API calls, subscriptions, timers, DOM manipulation
<code>componentDidUpdate()</code>	Respond to prop/state changes, conditional API calls
<code>componentWillUnmount()</code>	Cleanup timers, cancel requests, remove listeners
<code>shouldComponentUpdate()</code>	Performance optimization

Important Notes

- `render()` is the only **required** method
- Lifecycle methods run automatically at specific times
- **Don't call** `setState()` **in:** constructor, render, or `componentWillUnmount`

- **Safe to call** `setState()` **in:** `componentDidMount`, `componentDidUpdate` (with condition)
 - Modern React uses **Hooks** (like `useEffect`) instead of lifecycle methods in functional components
-

Pure Components in React

Pure Components are class components that implement a shallow comparison of props and state to determine if a re-render is necessary. They automatically optimize performance by preventing unnecessary re-renders when props and state haven't changed.

What Makes a Component "Pure"?

A pure component extends `React.PureComponent` instead of `React.Component`. It automatically implements `shouldComponentUpdate()` with a shallow prop and state comparison.

Regular Component vs Pure Component

Regular Component (`React.Component`):

- Re-renders every time parent re-renders, even if props/state are unchanged
- You must manually implement `shouldComponentUpdate()` for optimization

Pure Component (`React.PureComponent`):

- Automatically performs shallow comparison of props and state
- Only re-renders if there are actual changes
- Built-in performance optimization

Shallow Comparison

A **shallow comparison** checks if the reference to props/state has changed, not the deep content:

- Primitives (strings, numbers, booleans): compares values
 - Objects/Arrays: compares references (memory addresses)
-

Example

Here's a comprehensive example showing the difference:

```
import React, { Component, PureComponent } from 'react';

// Regular Component - Always re-renders
class RegularCounter extends Component {
  render() {
    console.log('RegularCounter rendered');
    return (
      <div style={{ border: '2px solid blue', padding: '10px', margin: '10px' }}>
        <h3>Regular Component</h3>
        <p>Count: {this.props.count}</p>
        <p>Name: {this.props.name}</p>
      </div>
    );
  }
}

// Pure Component - Only re-renders when props actually change
class PureCounter extends PureComponent {
  render() {
    console.log('PureCounter rendered');
    return (
      <div style={{ border: '2px solid green', padding: '10px', margin: '10px' }}>
        <h3>Pure Component</h3>
        <p>Count: {this.props.count}</p>
        <p>Name: {this.props.name}</p>
      </div>
    );
  }
}

// Parent Component
class App extends Component {
```

```

constructor(props) {
  super(props);
  this.state = {
    count: 0,
    name: 'John',
    unrelatedValue: 0
  };
}

incrementCount = () => {
  this.setState({ count: this.state.count + 1 });
}

incrementUnrelated = () => {
  // This doesn't change count or name
  this.setState({ unrelatedValue: this.state.unrelatedValue + 1 });
}

render() {
  console.log('Parent rendered');
  return (
    <div style={{ padding: '20px' }}>
      <h1>Pure Component Demo</h1>

      <button onClick={this.incrementCount}>
        Increment Count (Both will re-render)
      </button>

      <button onClick={this.incrementUnrelated} style={{
marginLeft: '10px' }}>
        Change Unrelated Value (Only Regular will re-render)
      </button>

      <p>Unrelated Value: {this.state.unrelatedValue}</p>

      <RegularCounter count={this.state.count} name={this

```



```

    s.state.name} />
    <PureCounter count={this.state.count} name={this.state.name} />

    <p style={{ marginTop: '20px', color: 'gray' }}>
      <strong>Check the console:</strong><br />
      - Click "Increment Count" → Both components re-render<br />
      - Click "Change Unrelated Value" → Only Regular component re-renders
    </p>
  </div>
);
}
}

export default App;

```

What Happens in the Example?

1. When you click "Increment Count":

- Parent state changes (count increases)
- Both RegularCounter and PureCounter receive new props
- Both components re-render (props actually changed)
- Console shows both rendered

2. When you click "Change Unrelated Value":

- Parent state changes (unrelatedValue increases)
- Parent re-renders
- RegularCounter re-renders (even though its props didn't change)
- PureCounter does NOT re-render (shallow comparison detects props are same)
- Console shows only RegularCounter rendered

Refs in Class Components

Refs (references) provide a way to directly access and interact with DOM elements or React components in the component tree. They allow you to bypass the typical React data flow and directly manipulate DOM nodes.

What are Refs?

Refs are used to:

- Access DOM elements directly (like focusing an input, measuring size, scrolling)
- Store mutable values that persist across renders (without triggering re-renders)
- Access child component instances

Creating Refs

In class components, refs are created using `React.createRef()` in the constructor.

When to Use Refs

✓ Good use cases:

- Managing focus, text selection, or media playback
- Triggering animations
- Integrating with third-party DOM libraries
- Measuring DOM element size or position

✗ Avoid using refs for:

- Things that can be done declaratively (use state instead)
- Accessing child component data (use props/callbacks instead)

```
import React from "react";

class FocusInput extends React.Component {
  // No constructor needed
  inputRef = React.createRef();
}
```

```

focusInput = () => {
  this.inputRef.current.focus();
};

render() {
  return (
    <div>
      <input
        ref={this.inputRef}
      />

      <button onClick={this.focusInput}>
        Focus Input
      </button>
    </div>
  );
}
}

export default FocusInput;

```

Error Boundaries:

Error Boundaries are React components that catch JavaScript errors anywhere in their child component tree, log those errors, and display a fallback UI instead of crashing the entire application.

What are Error Boundaries?

Error boundaries act like a JavaScript `try-catch` block but for React components. They prevent the entire app from breaking when an error occurs in one part of the UI.

Key Characteristics

- Error boundaries catch errors during **rendering**, in **lifecycle methods**, and in **constructors** of child components

- They are class components that implement specific lifecycle methods
 - Error boundaries **do NOT** catch errors in:
 - Event handlers (use regular try-catch)
 - Asynchronous code (setTimeout, promises)
 - Server-side rendering
 - Errors thrown in the error boundary itself
-

Creating an Error Boundary

An error boundary must implement one or both of these lifecycle methods:

1. `static getDerivedStateFromError(error)`

- Called during the render phase
- Used to update state so the next render shows fallback UI
- Must return a state object

2. `componentDidCatch(error, errorInfo)`

- Called during the commit phase
 - Used for logging error information
 - Receives error and component stack trace
-

Important: Error boundaries **must be class components** because the required lifecycle methods (`getDerivedStateFromError` and `componentDidCatch`) are not available in functional components. However, you can **wrap functional components** with error boundaries.

Real Example: Error Boundary Component

`ErrorBoundary.jsx`

```
import { Component } from "react";

class ErrorBoundary extends Component {
```

```

constructor(props) {
  super(props);
  this.state = { hasError: false };
}

static getDerivedStateFromError(error) {
  // Update state to show fallback UI
  return { hasError: true };
}

componentDidCatch(error, info) {
  // Log error to monitoring service
  console.error("Error caught:", error, info);
}

render() {
  if (this.state.hasError) {
    return <h2>Something went wrong 🙄</h2>;
  }

  return this.props.children;
}
}

export default ErrorBoundary;

```

Component That Crashes

BuggyCounter.jsx

```

function BuggyCounter({ count }) {
  if (count === 5) {
    throw new Error("Crashed at 5!");
  }

  return <h3>Count: {count}</h3>;
}

```

```
export default BuggyCounter;
```

Using Error Boundary in App

App.jsx

```
import { useState } from "react";
import ErrorBoundary from "../ErrorBoundary";
import BuggyCounter from "../BuggyCounter";

function App() {
  const [count, setCount] = useState(0);

  return (
    <>
      <ErrorBoundary>
        <BuggyCounter count={count} />
      </ErrorBoundary>

      <button onClick={() => setCount(count + 1)}>
        Increment
      </button>
    </>
  );
}

export default App;
```



What happens when `count === 5` ?

- 1 `BuggyCounter` throws an error
- 2 Error Boundary catches it
- 3 UI shows fallback instead of blank screen
- 4 App does NOT crash

Why Error Boundaries are needed

Without Error Boundary:

✗ Whole app crashes

With Error Boundary:

✓ Only broken part is replaced

Error Boundary vs Try-Catch

Error Boundary	Try-Catch
Catches render errors	Catches sync code only
Works in UI tree	No UI fallback
Class component	Works anywhere

✗ What Error Boundaries do NOT catch

```
<button onClick={() => {thrownewError("Not caught");}}>
```

✗ NOT caught

✓ Real-World Use Cases

- Dashboard widgets
- Charts
- Payment sections
- Third-party components

Wrap **only risky parts**, not whole app.

Interview questions & answers

What is an Error Boundary in React?

Answer:

An Error Boundary is a React component that catches JavaScript errors during rendering and lifecycle methods of its child components and displays a fallback UI instead of crashing the whole app.

Why are Error Boundaries needed?

Answer:

They prevent the entire application from crashing due to a single component failure, improving stability and user experience.

Which components can be Error Boundaries?

Answer:

Only **class components** can be Error Boundaries because they rely on lifecycle methods unavailable in functional components.

Which lifecycle methods are used in Error Boundaries?

Answer:

- `static getDerivedStateFromError`
 - `componentDidCatch`
-

What is the difference between `getDerivedStateFromError` and `componentDidCatch` ?

Answer:

`getDerivedStateFromError` updates state to render fallback UI, while `componentDidCatch` is used for logging error details.

Do Error Boundaries catch errors in event handlers or async code?

Answer:

No. Error Boundaries do not catch errors in event handlers, async code, promises, or `useEffect`.

Where should Error Boundaries be placed?

Answer:

Around **risky or isolated UI sections**, not around the entire application

Can functional components be Error Boundaries?

Answer:

No, not natively. Functional components do not support the required lifecycle methods.

What is a real-world use case for Error Boundaries?

Answer:

Wrapping third-party components like charts, payment widgets, or maps to prevent them from crashing the whole app.
